

课程须知

- 本课程适用软件版本: MWORKS.Syslab2024a
- 本课程示例运行需要软件首选项加载:
 - 基础库(TyBase)
 - 数学库(TyMath)
 - 图形库(TyPlot)

MWORKS.Syslab基础数学工具箱应用

刘尚

苏州同元软控信息技术有限公司

2024-01-12



TONGYUAN
Software and Control

@苏州同元软控信息技术有限公司版权所有，侵权必究
未经许可，不得复印、传播或以其他方式使用

目录

1. 基础数学工具箱功能概况

2. 基础数学与线性代数

3. 随机数生成与插值

4. 数值积分与微分方程

5. 傅里叶变换与数字滤波

1. 基础数学工具箱功能概述

1 基础数学

- 基本算术
- 三角学
- 离散数学
- 多项式
- 特殊函数

2 线性代数

- 线性方程
- 特征值奇异值
- 矩阵分解与运算
- 矩阵结构与属性

3 微分方程

- 常微分方程(ODE)
- 时滞微分方程(DDE)
- 边界值问题(BVP)
- 一维偏微分方程(PDE)

4 数值积分

- 表达式积分
- 数值数据积分
- 有限差分导数

5 插值

- 一维插值
- 网格插值
- 网格创建
- 散点插值(尽情期待)

6 随机数生成

- 随机数控制流
- 随机数生成

7 数字滤波与卷积

- 多维卷积
- 数字滤波

8 傅里叶变换

- 快速傅里叶变换
- 非均匀傅里叶变换
- 逆变换
- 零频平移

最新进展- 性能优化

- 持续性开展**84+71**个数学、统计库函数性能优化
- 针对性完成**62**个函数(数学核心库、插值库、微分方程库)性能(平均运行时间)和首次运行时间优化
- 部分工程用户代码性能优化与算法函数优化效果对比分析

interp1函数平均运行时间:

算法	平均Old/ μ s	平均New/ μ s	平均M/ μ s
cubic	12.6	3.575	862
linear	556.1	7.425	28
nearest	540	47.3	16
next	572	7.725	16
spline	64.6	28.3	18
pchip	21.4	6.12	34
previous	576.9	7.35	16

interp1函数首次运行时间:

算法	首次Old/s	首次New/s
cubic	6.654456	0.960867
linear	6.8334329	1.0394391
nearest	7.032450	0.876963
next	6.658025	0.881156
spline	6.694088	0.892971
pchip	7.196881	0.891746
previous	6.708036	0.889299



目录

1. 基础数学工具箱功能概况

2. 基础数学与线性代数

3. 随机数生成与插值

4. 数值积分与微分方程

5. 傅里叶变换与数字滤波

2.1 初等数学与线性代数

初等数学和线性代数板块提供基础科学计算函数：

- 初等数学函数包括支持算术运算（+、-、*、...）的功能、数学常量函数（Inf、pi、...）、多项式运算函数（poly、roots、...）以及特殊的数学函数
- 线性代数函数提供快速且数值稳健的矩阵计算。功能包括各种矩阵分解、线性方程求解、计算特征值或奇异值

一. 初等数学

三角学囊括大部分科学计算中使用的三角函数

函数类别	功能	数量
正弦	计算以弧或度为单位的正弦、反正弦、双曲、反双曲函数	7
余弦		7
正切		6
余割		6
正割		5
余切		5
斜边	平方和的平方根（斜边）	1
转换	度/弧度转换、坐标转换	6

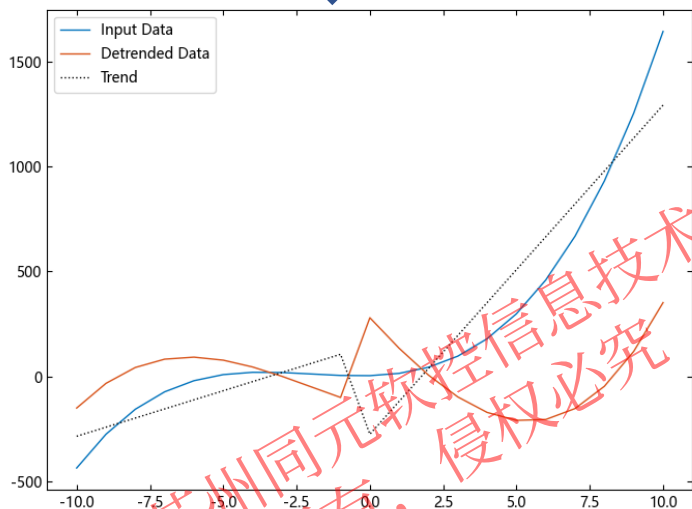
2.1 初等数学与线性代数

一. 初等数学

示例1: 去除多项式趋势(detrend):

```
t = -10:10;  
x = t.^3 + 6 * t.^2 + 4 * t + 3;  
bp = 0;  
y = detrend(x, 1, bp; SamplePoints=t,  
Continuous=false);  
plot(t, x, t, y, t, x - y, ":k")  
legend(["Input Data", "Detrended Data",  
"Trend"])
```

↓ 输出



提供质因数、组合、多项式处理**离散数学**、**多项式函数**

示例2: 部分分式展开(residue):

基本原理: 考虑次数分别为 n 和 m 的两个多项式 b 和 a 的分式 $F(s)$:

$$F(s) = \frac{b(s)}{a(s)} = \frac{b_n s^n + \dots + b_2 s^2 + b_1 s + b_0}{a_m s^m + \dots + a_2 s^2 + a_1 s + a_0}$$

分式 $F(s)$ 可以表示为几个简单分式之和。

$$\frac{b(s)}{a(s)} = \frac{r_m}{s - p_m} + \frac{r_{m-1}}{s - p_{m-1}} + \dots + \frac{r_0}{s - p_0} + k(s)$$

此总和称为 F 的部分分式展开式。值 $r_m, \dots, r_1$ 为残差, 值 $p_m, \dots, p_1$ 为极点, $k(s)$ 为 s 多项式。

实际案例: $F(s) = \frac{b(s)}{a(s)} = (2s^3 + s^2)/(s^3 + s + 1)$

```
b = [2, 1, 0, 0];  
a = [1, 0, 1, 1];  
r, p, k = residue(b, a)
```

```
julia> r  
3-element Vector{ComplexF64}:  
 0.5354179059386032 + 1.0389917087094538im  
 0.5354179059386032 - 1.0389917087094538im  
 -0.07083581187720643 + 0.0im  
  
julia> p  
3-element Vector{ComplexF64}:  
 0.3411639019140098 + 1.1615413999972526im  
 0.3411639019140098 - 1.1615413999972526im  
 -0.6823278038280195 + 0.0im  
  
julia> k  
1-element Vector{Float64}:  
 2.0
```

residue 返回代表部分分式展开式的复数根和极点, 以及常项 k :

$$F(s) = \frac{b(s)}{a(s)} = \frac{2s_3 + s_2}{s_3 + s + 1} = \frac{0.5354 + 1.0390im}{s - (0.3412 + 1.1615im)} + \frac{0.5354 - 1.0390im}{s - (0.3412 - 1.1615im)} + \frac{-0.0708}{s + 0.6823} + 2$$

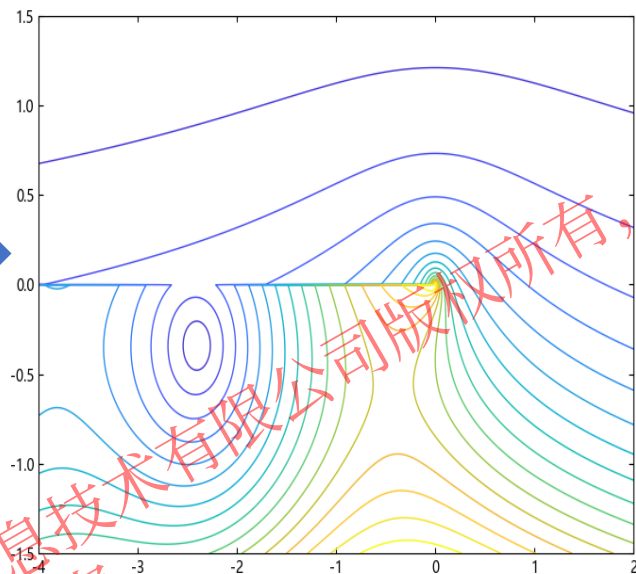
2.1 初等数学与线性代数

一. 初等数学

提供特殊函数、测试矩阵数学功能函数

示例3: Hankel函数的模数和相位(besselh):

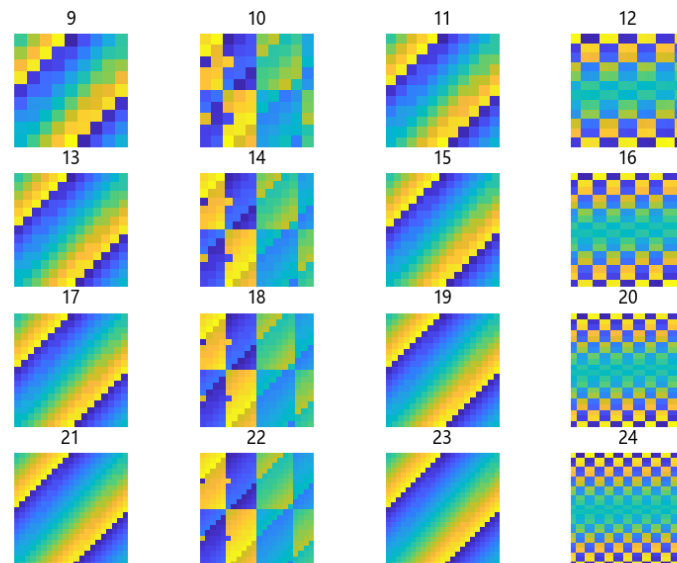
```
X, Y = meshgrid2(-4:0.002:2, -1.5:0.002:1.5)
Z = X + im * Y
H = zero(Z)
for i in 1:length(Z)
    if abs(Z[i]) == 0
        H[i] = 0
    else
        H[i] = besselh(0, Z[i])
    end
end
contour(X, Y, abs.(H), levels=0:0.2:3.2)
```



示例4: 幻方矩阵(magic):

使用 imagesc 观察 9 阶到 24 阶幻方矩阵的图案。这些图案表明 magic 使用三种不同的算法，取决于 $\text{mod}(n,4)$ 的值是 0、2 还是奇数。

```
using TyImages
for n = 1:16
    ax = subplot(4,4,n)
    ord = n+8
    m = magic(ord)
    imagesc(m)
    title("$ord")
    axis("equal")
    axis("off")
end
```



2.1.1 多项式

Syslab中将多项式表示为行向量，其中包含按降幂排序的系数。

$$p=[p_2, p_1, p_0] \xrightarrow{\text{表示}} p(x) = p_2x^2 + p_1x + p_0$$

#创建一个向量以表示二次多项式 $p(x) = x^2 - 4x + 4$

`p=[1, -4, 4]`



```
3-element Vector{Int64}:  
 1  
-4  
 4
```

#创建一个向量以表示二次多项式 $p(x) = 4x^5 - 3x^2 + 2x + 33$

`p = [4, 0, 0, -3, 2, 33]`



```
6-element Vector{Int64}:  
 4  
 0  
 0  
-3  
 2  
33
```

注意：

1. 多项式系数向量顺序由高到低
2. 多项式系数向量包含0次项系数，所以其长度为多项式最高次数加1
3. 必须将系数为0的多项式中间项输入到该向量中，因为0用作x的特定幂的占位符

2.1.1 多项式-多项式的四则运算

1. 多项式加减运算：系数向量相加减
2. 多项式乘法：使用`conv(p1,p2)`函数实现多项式相乘，`p1,p2`是两个多项式系数向量
3. 多项式除法：使用`q,r=deconv(p1,p2)`函数实现多项式相除，`q`为多项式`p1`除以`p2`的商式，`r`返回`p1`除以`p2`的余式，`q`和`r`仍是多项式系数向量。

#创建一个向量以表示二次多项式 $p1(x) = x^2 - 4x + 4$

`p1 = [1, -4, 4]`

3-element Vector{Int64}:

1
-4
4

#创建一个向量以表示二次多项式 $p2(x) = 3x^3 + 2x + 4$

`p2 = [3, 0, 2, 4]`

4-element Vector{Int64}:

3
0
2
4

#求解 $p1(x)+p2(x)$

`[0;p1]+p2`

4-element Vector{Int64}:

3
1
-2
8

#求解 $p1(x)-p2(x)$

`[0;p1]-p2`

4-element Vector{Int64}:

-3
1
-6
0

#求解 $p1(x)*p2(x)$

`conv(p1,p2)`

6-element Vector{Int64}:

3
-12
14
-4
-8
16

#求解 $p1(x)/p2(x)$

`q,r=deconv(p1,p2)`

`([0.0], [1, -4, 4])`

2.1.1 多项式-多项式的求导

Syslab中多项式求导使用polyder函数实现

1.返回 p 中的系数表示的多项式的导数

$$k = \text{polyder}(p) \longrightarrow k(x) = \frac{d}{dx} p(x)$$

2.返回多项式 a 和 b 的乘积的导数

$$k = \text{polyder}(p,b)[3] \longrightarrow k(x) = \frac{d}{dx} [a(x)b(x)]$$

3.返回多项式 a 和 b 的商的导数

$$q,d = \text{polyder}(a,b) \longrightarrow k(x) = \frac{d}{dx} \left[\frac{a(x)}{b(b)} \right]$$

#创建一个向量以表示二次多项式 $p1(x) = x^2 - 4x + 4$

```
julia> p1 = [1, -4, 4]
```

```
3-element Vector{Int64}:
```

```
1
```

```
-4
```

```
4
```

#对多项式求导 $q1(x) = 2x - 4$

```
julia> q1 = polyder(p)
```

```
2-element Vector{Int64}:
```

```
2
```

```
-4
```

#创建一个向量以表示二次多项式 $p2(x) = 2x + 4$,求解

$$q2(x) = \frac{d}{dx} [p1(x)p2(x)]$$

```
julia> p2 = [2, 4]; q2 = polyder(p1, p2)[3]
```

```
3-element Vector{Int64}:
```

```
6
```

```
-8
```

```
-8
```

#求解 $q, d = \frac{d}{dx} \left[\frac{p1(x)}{p2(x)} \right]$

```
#julia> q, d = polyder(p1, p2)
```

```
([0.5, 2.0, -6.0], [1.0, 4.0, 4.0])
```

2.1.1 多项式-多项式的求值

1.polyval函数根据特定值计算多项式（标量）

```
#创建一个向量以表示二次多项式 $p(x) = x^2 - 4x + 4$ 
julia> p = [1,-4,4]
3-element Vector{Int64}:
 1
-4
 4

#计算p(2)的值
julia> polyval(p,2)
0
```

2.Polyvalm函数以矩阵方式计算多项式

```
#创建一个向量以表示二次多项式 $p(X) = X^2 - 4X + 4$ 
julia> p = [1,-4,4]
3-element Vector{Int64}:
 1
-4
 4

#创建方阵X
julia> X = [2 4 5; -1 0 3; 7 1 5]
3×3 Matrix{Int64}:
 2  4  5
-1  0  3
 7  1  5

#计算p(2)的值
julia> Y = polyvalm(p,X)
3×3 Matrix{Int64}:
31  -3  27
23   3  -2
20  29  47
```

2.1.1 多项式-多项式的求根

Syslab中，使用roots函数对多项式求根。
roots(p):以列向量的形式返回 p 表示的多项式的根

#对方程 $3x^2-2x-4=0$ 求解

```
p = [3, -2, -4];  
r = roots(p)
```

```
2-element Vector{Float64}:  
 1.5351837584879962  
 -0.8685170918213299
```

#对方程 $x^4-1=0$ 求解

```
p = [1, 0, 0, 0, -1];  
r = roots(p)
```

```
4-element Vector{ComplexF64}:  
 -1.0000000000000004 + 0.0im  
 8.326672684688674e-17 + 0.9999999999999996im  
 8.326672684688674e-17 - 0.9999999999999996im  
 0.9999999999999999 + 0.0im
```


2.2 初等数学与线性代数

初等数学和线性代数板块提供基础科学计算函数：

- 初等数学函数包括支持算术运算（+、-、*、...）的功能、数学常量函数（lnf、pi、...）、多项式运算函数（poly、roots、...）以及特殊的数学函数
- **线性代数**函数提供快速且数值稳健的矩阵计算。功能包括各种矩阵分解、线性方程求解、计算特征值或奇异值

二. 线性代数

函数分类	功能	典型函数
线性方程	提供线性方程组多种求解方法	lsqminnorm、linsolve
特征值和奇异值	特征值和特征向量的计算和奇异值分解。	eigen、svd
矩阵分解	常用矩阵分解（Cholesky、LU、QR）	lu、cholesky
矩阵运算	提供以矩阵为对象的运算函数	sqrt、dot
矩阵结构	判断、提取、构造矩阵结构的特殊属性	Issymmetric、istril
矩阵属性	矩阵自带数值属性	norm、rank

2.2.1 线性方程求解

类别	函数	功能描述
线性方程	\	对线性方程组 $Ax = B$ 求解 x
	/	对线性方程组 $xA = B$ 求解 x
	linsolve	对线性方程组求解
	inv	矩阵求逆
	pinv	Moore-Penrose 伪逆
	lscov	存在已知协方差情况下的最小二乘解
	sylvester	求 Sylvester 方程 $AX + XB = C$ 的 X 解

线性方程求解: $X=\text{linsolve}(A,B)$ 使用以下方法之一求解线性方程组 $AX = B$:

- 当 A 是方阵时, linsolve 使用 LU 分解和部分主元消去法。
- 对于所有其他情况, linsolve 使用 QR 分解和列主元消去法

```
julia> A = magic(3)
3×3 Matrix{Int64}:
 8  1  6
 3  5  7
 4  9  2

julia> B = [15; 15; 15]
3-element Vector{Int64}:
 15
 15
 15

julia> x =linsolve(A,B)
3-element Vector{Float64}:
 0.9999999999999999
 1.0
 1.0000000000000002
```

矩阵求逆:对于一个方阵 A , 如果存在一个与其相同的方阵 B , 使得 $AB=BA=I$ (I 为单位矩阵), 则称 B 为 A 的逆矩阵, 当然, A 于是 B 的逆矩阵

- Inv(A):**求方阵 A 的逆矩阵

```
julia> A=magic(3)
3×3 Matrix{Int64}:
 8  1  6
 3  5  7
 4  9  2

julia> B=inv(A)
3×3 Matrix{Float64}:
 0.147222 -0.144444  0.0638889
-0.0611111  0.0222222  0.105556
-0.0194444  0.188889 -0.102778

julia> C=A*B;ty_format("short",C)
 1.0000  0.0000 -0.0000
-0.0000  1.0000  0.0000
 0.0000  0.0000  1.0000
```

其他特征值与奇异值相关函数使用详见Syslab帮助文档

2.2.1 线性方程组求解

矩阵的方法解线性方程组：

$$\begin{cases} x + 2y + 3z = 5 \\ x + 4y + 9z = -2 \\ x + 8y + 27z = 6 \end{cases} \quad \Rightarrow \quad A = \begin{bmatrix} 1 & 2 & 3 \\ 1 & 4 & 9 \\ 1 & 8 & 27 \end{bmatrix}, b = \begin{bmatrix} 5 \\ -2 \\ 6 \end{bmatrix}, X = \begin{bmatrix} x \\ y \\ z \end{bmatrix}$$

线性方程组 $Ax = b$ 的求解：

- A 满秩， $Ax = b$ 称为恰定方程，有唯一解 x ，一般用 $x = A \backslash b$ 求解速度更快。
- 当右端向量 $b = 0$ 时，称为齐次方程组，总有零解，称解 $x = 0$ 为平凡解。 A 的秩小于 n ，无穷多个非平凡解，通解中包含 $n - \text{rank}(A)$ 个线性无关的解向量，用函数 $\text{null}(A, \text{"rational"})$ 可求得基础解系。
- 当右端向量 $b \neq 0$ 时：
 - 1) 当 $\text{rank}(A) = \text{rank}([A, b]) = n$ 时，方程组有唯一解： $x = A \backslash b$ 或 $x = \text{pinv}(A) * b$ 。
 - 2) 当 $\text{rank}(A) = \text{rank}([A, b]) < n$ 时，有无穷多个解，通解由一个特解和对应的齐次方程组 $Ax = 0$ 的通解组成。用 $A \backslash b$ 求特解，用 $\text{null}(A, \text{"rational"})$ 求通解。
 - 3) 当 $\text{rank}(A) < \text{rank}([A, b])$ ，无解，可求 x 使得取最小值，其解为 $x = A \backslash b$ 或 $x = \text{pinv}(A) * b$ 。

2.2.1 线性方程组求解

$X = \text{linsolve}(A,B)$ 使用以下方法之一求解线性方程组 $AX = B$:

- 当 A 是方阵时, `linsolve` 使用 LU 分解和部分主元消去法。
- 对于所有其他情况, `linsolve` 使用 QR 分解和列主元消去法。

```
julia> A=[1 2 3;1 4 9;1 8 27]
3×3 Matrix{Int64}:
```

```
 1  2  3
 1  4  9
 1  8 27
```

```
julia> B=[5,-2, 6]
3-element Vector{Int64}:
```

```
 5
-2
 6
```

Tips: 输入为方阵。如果矩阵未正确缩放或接近奇异矩阵, `linsolve` 计算的数值将不准确。使用 `rcond` 或 `cond` 检查矩阵的条件数。

```
julia> x =linsolve(A,B)
3-element Vector{Float64}:
 23.0
-14.5
 3.6666666666666665
```

2.2.2 矩阵求逆

在Syslab中，求方阵A的逆矩阵可调用函数`inv(A)`。

则原线性方程组可简化为： $Ax=b$

其解为 $x = A^{-1}b$

方程的解采用矩阵的逆运算或者采用左除的运算进行求解

- 采用逆运算： $X=\text{inv}(A)*B$
- 采用左除 $X=A\backslash B$

Tips:

- 1.从执行时间和数值准确性方面而言，一种更好的方法是使用矩阵反斜杠运算符，即 $x = A\backslash b$ 。这会使用高斯消去法求解，而不必显式构造逆矩阵。
- 2.Julia中：如A为奇异矩阵，则打印错误:SingularException(3)

```
julia> A=[1 2 3;1 4 9;1 8 27]  
3×3 Matrix{Int64}:
```

```
1 2 3  
1 4 9  
1 8 27
```

```
julia> B=[5,-2, 6]  
3-element Vector{Int64}:
```

```
5  
-2  
6
```

```
julia> X=A\B  
3-element Vector{Float64}:
```

```
23.0  
-14.5  
3.6666666666666665
```

```
julia> X=inv(A)*B  
3-element Vector{Float64}:
```

```
23.0  
-14.5  
3.6666666666666667
```

2.2.3 矩阵求伪逆

Moore-Penrose 伪逆是一种矩阵，可在不存在逆矩阵的情况下作为逆矩阵的部分替代。此矩阵常被用于求解没有唯一解或有无数解的线性方程组。此时使用 `pinv(A)`

定义一个奇异矩阵A

$$A = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix}$$

对该矩阵求逆，打印错误信息

对该矩阵求伪逆，得到结果

```
julia> A=[1 2 3; 4 5 6; 7 8 9]
3×3 Matrix{Int64}:
 1  2  3
 4  5  6
 7  8  9

julia> inv(A)
ERROR: SingularException(3)

julia> pinv(A)
3×3 Matrix{Float64}:
-0.638889  -0.166667   0.305556
-0.0555556  1.2837e-16   0.0555556
 0.527778   0.166667  -0.194444
```

2.2.4 特征值和奇异值

类别	函数	功能描述
特征值与奇异值	eigen	计算 A 的特征值分解，返回特征值与特征向量
	eigvals	返回A的特征值
	eigvecs	计算 A 的特征值向量
	svd	计算 A 的奇异值分解 (SVD) 并返回一个 SVD 对象。
	svdvals	按降序返回 A 的奇异值
	ordeig	拟三角矩阵的特征值
	polyeig	多项式特征值问题
	hessenberg	矩阵的 Hessenberg 形式
	schur	Schur分解
	schur1	Schur分解
	rsf2csf	将实数Schur形式变换为复数Schur形式
	cdf2rdf	拟三角矩阵的特征值

2.2.4 特征值和奇异值-特征值

A是一个 $n \times n$ 的矩阵，若存在非零向量 x 和实数 λ 使得 $Ax = \lambda x$ ，则 x 和 λ 分别为 A 的特征向量和对应的特征值。特征值的个数等于 A 的阶数。

特征向量：一个向量经过线性变换，仍留在它所张成的空间中

特征值：描述特征向量经过线性变换后的缩放程度

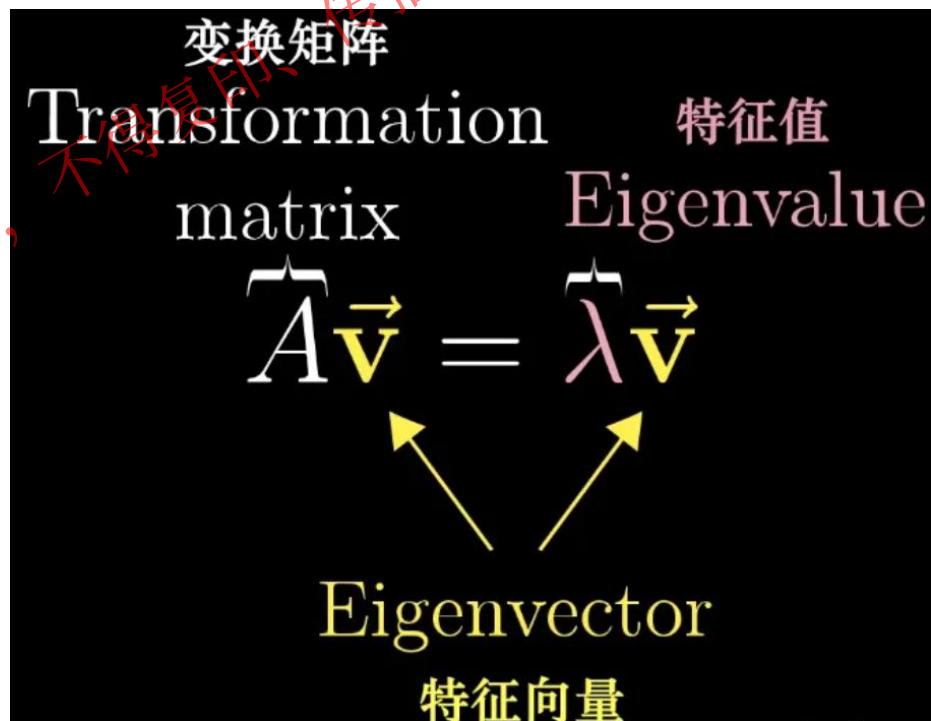
Syslab的特征值函数为 **eigvals**，特征向量函数 **eigvecs**

(1) **E=eigvals(A)**：求矩阵 A 的全部特征值，构成向量 E

(2) **E=eigvecs(A)**：将返回矩阵 A 特征向量

并非所有的矩阵都存在特征向量

如果 V 是非奇异的，这将变为特征值分解。



2.2.4 特征值和奇异值-奇异值分解

设 A 为 $m \times n$ 阶矩阵, $q = \min(m, n)$, $A^T A$ 的 q 个非负特征值的算术平方根叫作 A 的奇异值。

奇异值分解是线性代数和矩阵论中一种重要的矩阵分解法, 能适用于任意的矩阵, 在信号处理、统计学、机器学习等领域有着广泛的应用。

SVD可以帮助我们揭示组成该线性变换的最本质的变换, 具体地, SVD揭示了这样一个事实: **对于任意的矩阵 A , 我们总能找到一组单位正交基, 使得 A 对其进行变换之后, 得到的向量组仍然是正交的。**

$A = U \sum V^T$, U 是正交矩阵, $\sum$ 是对角矩阵, V 是正交矩阵
 U 是 $M \times M$ 矩阵, $U = AA^T$, U 是正交矩阵, 即满足 $U^T U = I$
 $\sum$ 是 $M \times N$ 矩阵, 除了主对角线全部为0, 主对角线每个元素为奇异值
 V^T 是 $N \times N$ 矩阵, $V^T = A^T A$, V^T 是正交矩阵, 即满足 $V^T V = I$

```
#计算A的"满"奇异值分解
svd(A; full = false)
#计算 A 和 B 的广义奇异值分解
svd(A, B)
```

```
julia> A = [1 2; 3 4; 5 6; 7 8]
4×2 Matrix{Int64}:
 1  2
 3  4
 5  6
 7  8
```

```
julia> F = svd(A, full = true)
SVD{Float64, Float64, Matrix{Float64}},
```

```
U factor:
4×4 Matrix{Float64}:
-0.152483 -0.822647 -0.394501 -0.379959
-0.349918 -0.421375  0.242797  0.800656
-0.547354 -0.0201031 0.69791  -0.461434
-0.744789  0.381169 -0.546205  0.0407376
```

```
singular values:
2-element Vector{Float64}:
 14.269095499261486
  0.6268282324175424
```

```
Vt factor:
2×2 Matrix{Float64}:
-0.641423 -0.767187
 0.767187 -0.641423
```

2.2.5 矩阵分解

矩阵分解 (decomposition, factorization)是将矩阵拆解为数个矩阵的乘积，可分为三角分解、满秩分解、QR分解、Jordan分解等。

- 常见的有三种：

三角分解法 (Triangular Factorization)、QR 分解法 (QR Factorization)、L-U分解法 (L-U Factorization)

函数名	简介
lu	LU 矩阵分解
ldlt	实对称三对角矩阵 S 的 LDLt 分解
cholesky	Cholesky 分解
cholupdate	Cholesky 分解的秩 1 更新
qrdelete	从 QR 分解中删除列或行
qrinsert	将列或行插入 QR 分解
qrupdate	QR 分解的秩 1 更新
planerot	Givens 平面旋转

2.2.5 矩阵分解-LU分解

矩阵的 LU 分解就是将一个矩阵表示一个可交换下三角矩阵和一个上三角矩阵的乘积形式。
(方阵 A 非奇异, $\det(A) \neq 0$, LU 分解总是可以进行的。)

```
using TyMath
A = [10 -7 0
     -3  2 6
      5 -1 5]
L,U = lu(A,NoPivot())
```



```
L,U = lu(A)
L,U,P = lu(A)
m=diagm([1,1,1])
P=m[P,:]
P'*L*U
```

```
LU{Float64, Matrix{Float64}, Vector{Int64}}
```

L factor:

3×3 Matrix{Float64}:

```
1.0  0.0  0.0
-0.3 1.0  0.0
0.5 -25.0 1.0
```

U factor:

3×3 Matrix{Float64}:

```
10.0 -7.0  0.0
 0.0 -0.1  6.0
 0.0  0.0 155.0
```

```
julia> L*U
```

3×3 Matrix{Float64}:

```
10.0 -7.0  0.0
-3.0  2.0  6.0
 5.0 -1.0  5.0
```

在Syslab2023b0330之后的版本中提供的lu分解会对矩阵是否为奇异矩阵进行检查, 如果存在奇异矩阵会报错

2.2.5 矩阵分解-qr分解

对矩阵 A 进行 QR 分解，就是把 A 分解为一个正交矩阵 Q 和一个上三角矩阵 R 的乘积形式， QR 分解只能对方阵进行。调用格式： $QR = \text{qr}(A)$ 。

```
julia> A=magic(5)
5×5 Matrix{Int64}:
 17  24   1   8  15
 23   5   7  14  16
  4   6  13  20  22
 10  12  19  21   3
 11  18  25   2   9
```

```
julia> qr(A)
LinearAlgebra.QRCompactWY{Float64, Matrix{Float64}},
Matrix{Float64}}
Q factor:
5×5 LinearAlgebra.QRCompactWYQ{Float64, Matrix{Float64}},
Matrix{Float64}}:
-0.523387  0.505754  0.67347  -0.121542  -0.0441038
-0.708111 -0.696573 -0.0177274  0.0815416  -0.0800023
-0.12315  0.136742 -0.355751  -0.630744  -0.664635
-0.307875  0.191057 -0.412231  -0.424672  0.720021
-0.338662  0.451439 -0.499631  0.632767  -0.177441
R factor:
5×5 Matrix{Float64}:
-32.4808 -26.6311 -21.3973 -23.7063 -25.8615
  0.0    19.8943  12.3234  1.94393  4.08558
  0.0    0.0    -24.3985 -11.6316 -3.74148
  0.0    0.0    0.0    -20.0982 -9.97394
  0.0    0.0    0.0    0.0    -16.0005
```

2.2.6 矩阵结构

类别	函数	功能描述
矩阵结构	tril	矩阵的下三角形部分
	triu	矩阵的上三角形部分
	bandwidth	矩阵的上下带宽
	isbanded	确定矩阵是否在特定带宽范围内
	isdiag	确定矩阵是否为对角矩阵
	ishermitian	确定矩阵是 Hermitian 矩阵
	issymmetric	确定矩阵是对称矩阵
	istril	确定矩阵是否为下三角矩阵
	istriu	确定矩阵是否为上三角矩阵

```
julia> a=[1 2 3;4 5 6;7 8 9]
3×3 Matrix{Int64}:
 1  2  3
 4  5  6
 7  8  9

julia> b=tril(a)
3×3 Matrix{Int64}:
 1  0  0
 4  5  0
 7  8  9

julia> c=triu(a)
3×3 Matrix{Int64}:
 1  2  3
 0  5  6
 0  0  9

julia> d=bandwidth(a)
(2, 2)

julia> isbanded(a,2,2)
true
```

```
julia> isdiag(a)
false

julia> ishermitian(a)
false

julia> issymmetric(a)
false

julia> d=bandwidth(a)
(2, 2)

julia> isbanded(a,2,2)
true

julia> istril(b)
true

julia> istriu(c)
true
```


2.2.7 矩阵属性

类别	函数	功能描述
矩阵属性	det	矩阵行列式
	rank	矩阵的秩
	tr	对角线元素之和
	norm	计算范数
	cond	矩阵条件数
	condskeel	相对条件数
	condeig	与特征值有关的条件数
	rref	简化的行阶梯形矩阵 (Gauss-Jordan 消元法)
	subspace	两个子空间之间的角度

1.方阵行列式：把一个方阵看做一个行列式，并对其按行列式的规则求值，这个值就称为对应的行列式的值

- **det(A)**:求方阵A所对应的行列式的值

```
julia> A=[1 3 2;-3 2 1;4 1 2]
3×3 Matrix{Int64}:
 1  3  2
-3  2  1
 4  1  2
julia> B=det(A)
11.0
```

2.矩阵的秩：矩阵线性无关的行数或列数称矩阵的秩

- **rank(A)**:求矩阵A的秩

```
julia> A = [1 2 3; 3 4 5;4 5 6]
3×3 Matrix{Int64}:
 1  2  3
 3  4  5
 4  5  6
julia> B=rank(A)
2
```

3.矩阵的迹：矩阵的迹等于矩阵的对角线元素之和，也等于矩阵的特征值之和

- **tr(A)**:求矩阵A的迹

```
julia> A = [1 2 3; 3 4 5;4 5 6]
3×3 Matrix{Int64}:
 1  2  3
 3  4  5
 4  5  6
julia> B=tr(A)
11
```

4.矩阵的范数：矩阵的范数用来度量矩阵在某种意义下的长度

- **norm(A,p)**:求矩阵A的范数，p可取1,2, Inf, -Inf, 默认为2

```
julia> A = [1 2 3; 3 4 5;4 5 6]
3×3 Matrix{Int64}:
 1  2  3
 3  4  5
 4  5  6
julia> B=norm(A)
11.874342087037917
```

5.矩阵的条件数：矩阵A的条件数等于A的范数与A的逆矩阵的范数的乘积，条件数与接近1，矩阵的性能越好，反之，矩阵的性能越差。

- **cond(A,p)**:求方阵A的p范数条件数

```
julia> A=magic(3);B=cond(A,1)
5.333333333333333
julia> B=cond(A,2)
4.330127018922193
julia> B=cond(A,Inf)
5.333333333333333
```



目录

1. 基础数学工具箱功能概况

2. 基础数学与线性代数

3. 随机数生成与插值

4. 数值积分与微分方程

5. 傅里叶变换与数字滤波

3.1 随机数生成

随机数生成板块内函数提供伪随机数序列生成、随机置换整数向量、结果的可重复性控制、随机流控制等功能函数

提供多种伪随机数、种子等算法函数

函数名称	功能
rand	均匀分布随机数
randn	正态分布随机数
randi	均匀分布的伪随机整数
randperm	随机序列
randpermk	随机整数序列
sprand	稀疏均匀分布随机矩阵

函数名称	算法
MCG16807	乘法同余生成器
MT19937ar	梅森旋转
DSFMT19937	面向 SIMD 的快速梅森旋转算法
SHR3	移位寄存器生成器与线性同余生成器求和
SWB2712	修正的借位减法生成器

3.1 随机数生成

函数名称	功能	函数接口
rand	均匀分布随机数	rand(rng=default_rng(), S, dims)
randn	正态分布随机数	randn(rng=default_rng(), T, dims)
randi	均匀分布的伪随机整数	randi(rng=default_rng(), interval, dims)
randperm	随机序列	randperm(rng=default_rng(), n)
randpermk	随机整数序列	randpermk(rng=default_rng(), n, k)
sprand	稀疏均匀分布随机矩阵	sprand(rng=default_rng(), m, n, p)

每一个函数都有一个共同的接口rng,用于设置随机数种子。
不传入随机数种子则每次都生成不同的随机数，传入则可以控制随机数生成结果。



3.1.1 均匀分布随机数

使用rand函数控制均匀分布的随机数生成

示例一：生成3×4数组

```
a = rand(3,4)
```

示例二：指定数据类型生成3×4数组

```
a = rand(ComplexF64,3,4)  
b = rand(UInt64,3,4)
```

示例三：控制随机数流

利用此方法可以满足以下需求

- 1.重复运行脚本，生成随机数相同
- 2.随机数生成时种子状态可以保存、复用方便工程验证

```
rng1 = MT19937ar(5489)  
rng2 = MT19937ar(5489)  
for i = 1:10  
    global a1 = rand(rng1,3,4)  
    global a2 = rand(rng2,3,4)  
end  
a1 == a2
```

3.2 插值

■插值板块内函数提供一维二维及更高维网格插值函数、网格创建等通过已知数据点添加新数据点的算法函数

插值方法

函数名称	功能
interp1	一维数据插值（表查找）
interp2	meshgrid 格式的二维网格数据的插值
interp3	meshgrid 格式的三维网格数据的插值
interpN	ndgrid 格式的N 维网格数据的插值
griddedInterpolant	网格数据插值

插值算法

算法名称	算法
cubic	三次卷积插值
makima	修正Akima三次 Hermite 插值
spline	三次样条数据插值
pchip	分段三次 Hermite 插值多项式

3.2.1 插值-三次样条

三次样条是实际中最常用的样条。而三次样条凭借其能给出满足光滑性要求的最简单表示，成为最受欢迎的样条。

线性样条的缺点：不光滑，两个样条的交点处（结点）斜率发生了剧烈不变化。

二次样条的缺点：最前面两个点是由直线连接的，最后一个区间中的样条摆动过大。

四次或更高次样条表现出高次多项式本身固有的不稳定性，所以我们一般不使用。

利用syslab内置函数spline 可以很容易地算出三次样条。它的一般用法为：

s= spline(x,y,xq)

返回与 xq 中的查询点对应的插值 s 的向量。s 的值由 x 和 y 的三次样条插值确定。

3.2.1 插值-三次样条

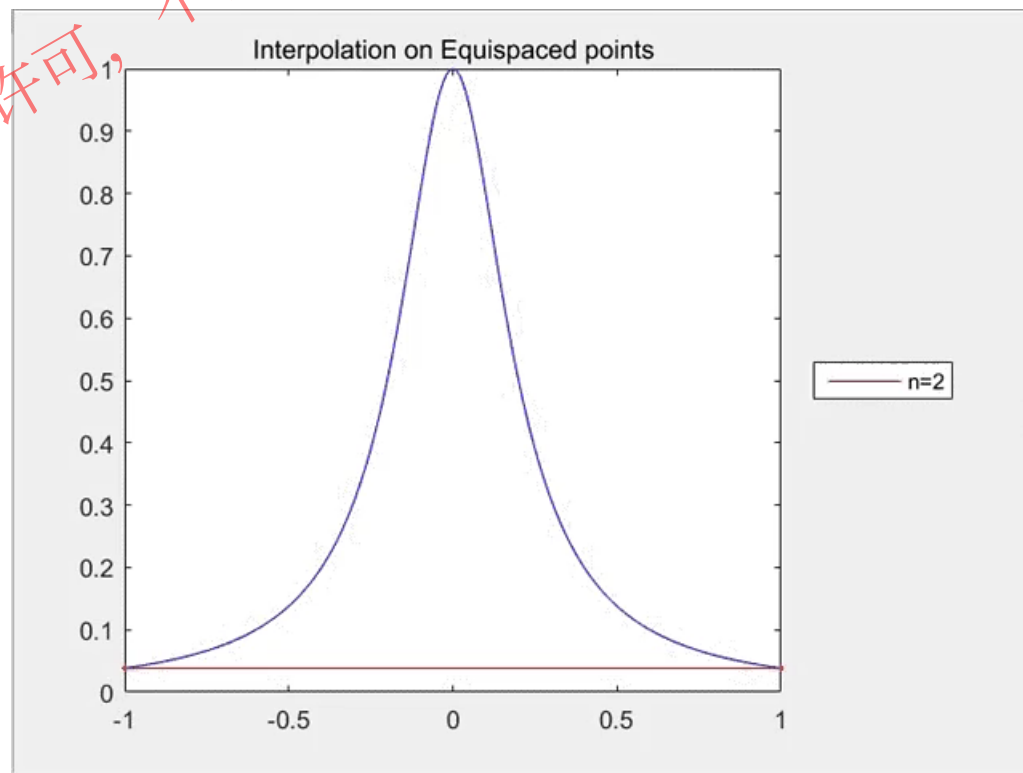
龙格现象

在科学计算领域，龙格现象（Runge）指的是对于某些函数，使用均匀节点构造高次多项式差值时，在插值区间的边缘的误差可能很大的现象。

它是由Runge在研究多项式差值的误差时发现的，表明：**并不是插值多项式的阶数越高，效果就会越好。**

最经典的一个例子是在均匀节点上对龙格函数

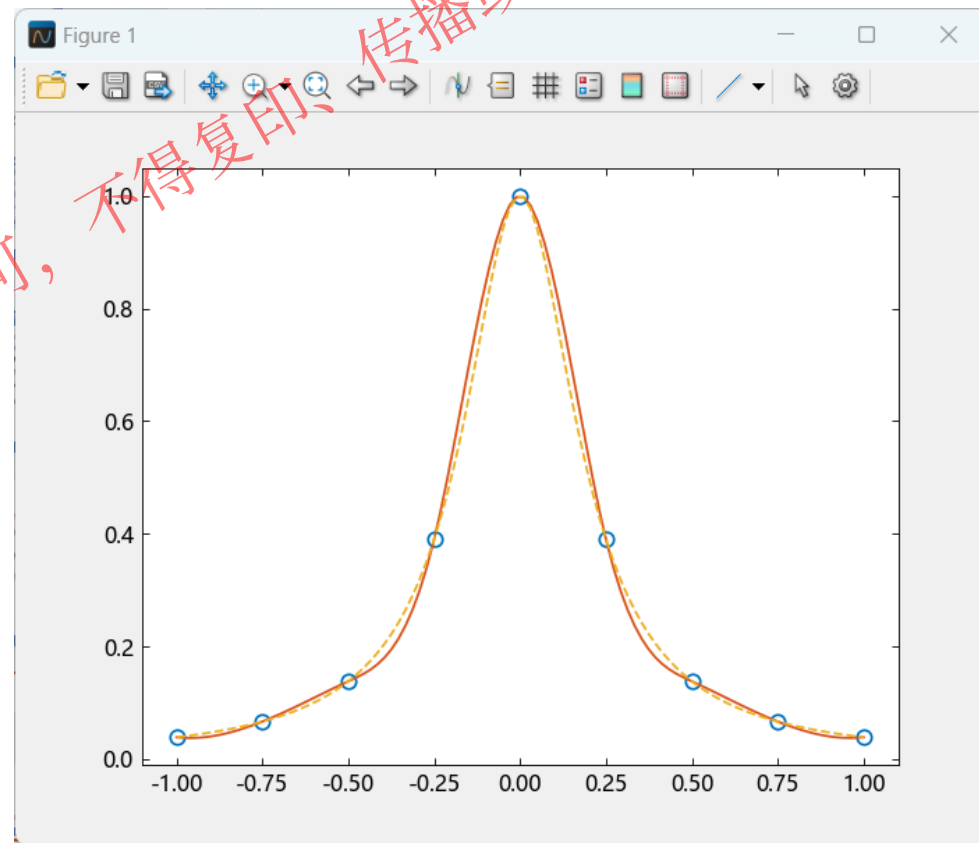
$$f(x) = \frac{1}{1 + 25x^2}, x \in [-1, 1]$$



3.2.1 插值-三次样条

a.非节点样条

```
x=LinRange(-1,1,9)
y= @. 1/(1+25*x^2)
xx=LinRange(-1,1,100)
yy=spline(x,y,xx)
yr=@. 1/(1+25*xx^2)
plot(x,y,"o",xx,yy,xx,yr,"--")
```



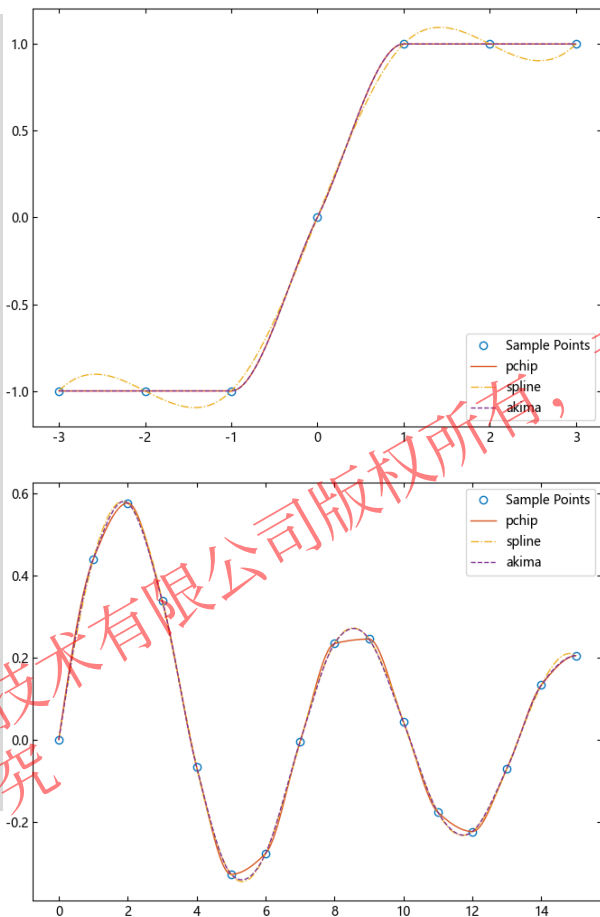
非结点样条可以很好地模拟龙格函数的图形，结点之间并没有产生大的震荡

3.2 插值

多维度、多算法、多接口的插值算法

示例1: 使用spline、pchip、makima进行数据插值:

```
x = [-3,-2,-1,0,1,2,3];
y = [-1, -1, -1, 0, 1, 1, 1];
xq1 = -3:0.01:3;
p = interp1(x, y, xq1, "pchip");
s = interp1(x, y, xq1, "spline");
m = interp1(x, y, xq1, "makima");
plot(x, y, "o", xq1, p, "-", xq1, s,
     "--", xq1, m, "--")
legend(["Sample Points", "pchip",
       "spline", "akima"], loc = "southeast")
figure(2)
x = 0:15;
y = besselj.(1, x);
xq2 = 0:0.01:15;
p = interp1(x, y, xq2, "pchip");
s = interp1(x, y, xq2, "spline");
m = interp1(x, y, xq2, "makima");
plot(x, y, "o", xq2, p, "-", xq2, s,
     "--", xq2, m, "--")
legend(["Sample Points", "pchip",
       "spline", "akima"])
```

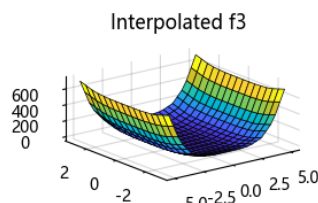
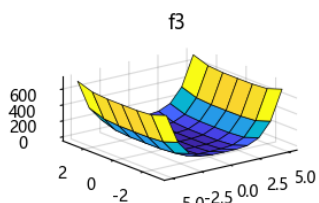
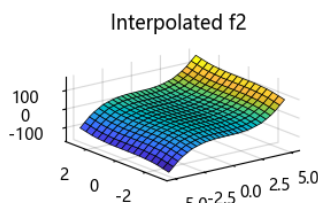
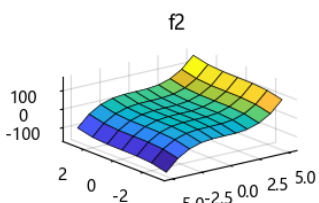
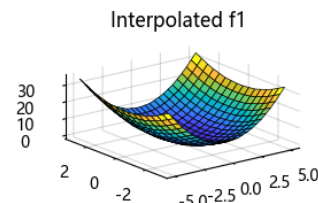
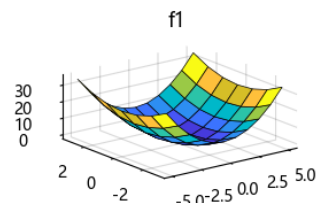


示例2: 为 x 域范围外的所有查询点指定常量值:

使用 griddedInterpolant 在相同的查询点上进行三组不同值插值。

使用 $-5 \leq X \leq 5$ 和 $-3 \leq Y \leq 3$ 创建一个采样点网格。

```
gx = -5:5;
gy = -3:3;
X, Y = ndgrid(gx, gy)
f1 = @. X^2 + Y^2;
f2 = @. X^3 + Y^3;
f3 = @. X^4 + Y^4;
V = cat(f1, f2, f3, dims=3);
F = griddedInterpolant(X, Y, V)
qx = -5:0.4:5;
qy = -3:0.4:3;
XQ, YQ = ndgrid(qx, qy)
VQ = F(XQ, YQ)
subplot(3, 2, 1); surf(X, Y, f1); title("f1")
subplot(3, 2, 2); surf(XQ, YQ, VQ[:, :, 1]); title("Interpolated f1")
subplot(3, 2, 3); surf(X, Y, f2); title("f2")
subplot(3, 2, 4); surf(XQ, YQ, VQ[:, :, 2]); title("Interpolated f2")
subplot(3, 2, 5); surf(X, Y, f3); title("f3")
subplot(3, 2, 6); surf(XQ, YQ, VQ[:, :, 3]); title("Interpolated f3")
tightlayout()
```

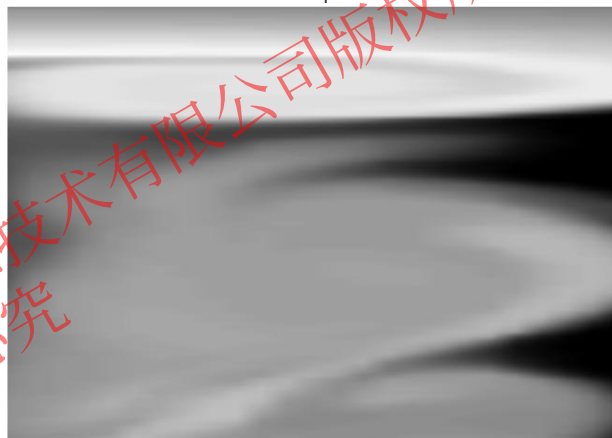
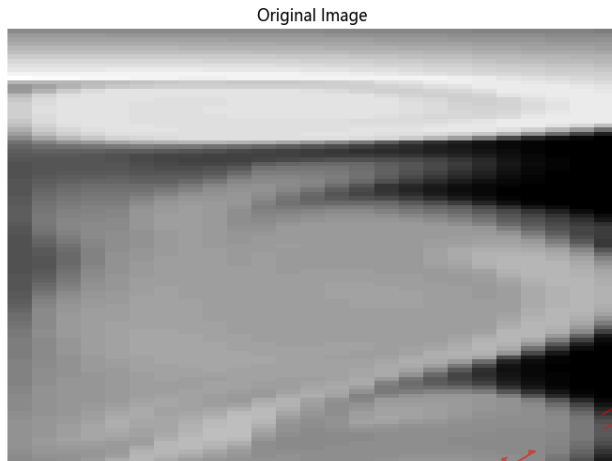


3.2 插值

多维度、多算法、多接口的插值算法

示例3: 灰度图像优化:

```
include(pkgdir(TyMath)*"/examples/do
cs/flujet.jl")
V = X[200:300, 1:25]
figure()
ims = imagesc(reverse(V, dims = 2),
xvalue = [0 125], yvalue = [100 0])
colormap(ims, "gray")
axis("off")
title("Original Image")
figure()
Vq = interp2(V,5)
ims = imagesc(reverse(Vq,dims=2),
xvalue = [0 125], yvalue = [100 0])
colormap(ims, "gray")
axis("off")
title("Linear Interpolation")
```

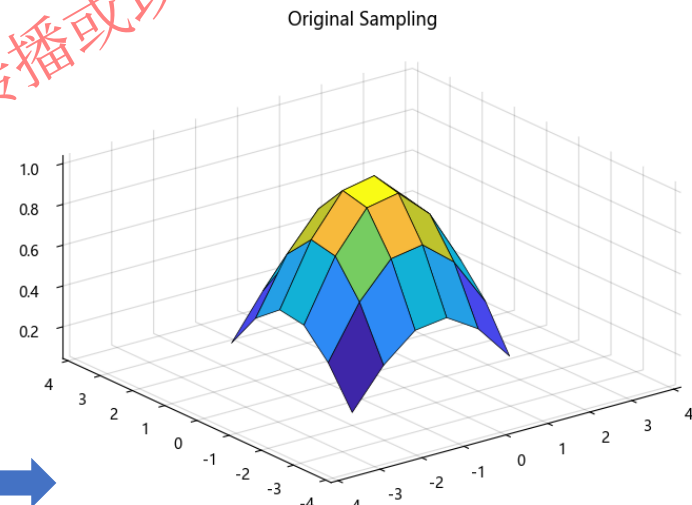


示例4: 在 X 和 Y 域范围外计算:

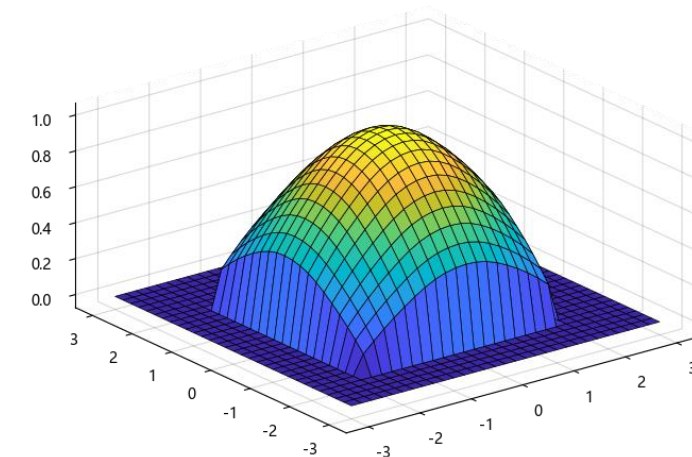
在 $[-2, 2]$ 的范围内从两个维度对函数进行粗略采样。

```
X, Y = meshgrid2(-2:0.75:2, -
2:0.75:2)
R = @. sqrt(X^2 + Y^2) + $eps()
V = @. sin(R) / R
figure()
surf(X, Y, V)
xlim([-4 4])
ylim([-4 4])
title("Original Sampling")
Xq, Yq = meshgrid2(-3:0.2:3, -
3:0.2:3)
Vq = interp2(X, Y, V, Xq, Yq,
"cubic", 0)
```

```
figure()
surf(Xq, Yq, Vq)
title("Cubic Interpolation with
Vq=0 Outside Domain of X and Y")
```



Cubic Interpolation with Vq=0 Outside Domain of X and Y



目录

1. 基础数学工具箱功能概况

2. 基础数学与线性代数

3. 随机数生成与插值

4. 数值积分与微分方程

5. 傅里叶变换与数字滤波

4.1 数值积分与微分方程

- 数值积分类板块函数提供无论函数表达式是否已知都可以求积分的近似值的解决方案
- 微分方程板块函数提供ODE(常微分方程)、DDE(时滞微分方程)、BVP(边界值问题)、SDE(随机微分方程)等微分方程求解的解决方案

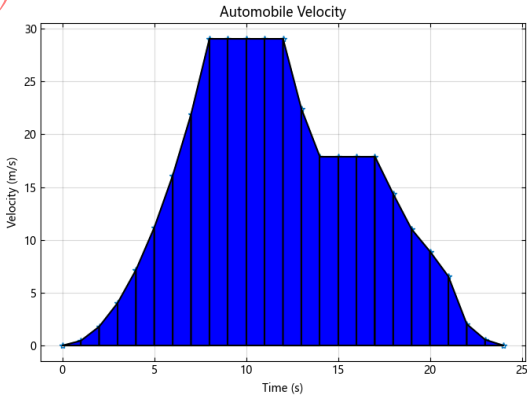
一. 数值积分

求积、二重积分和三重积分

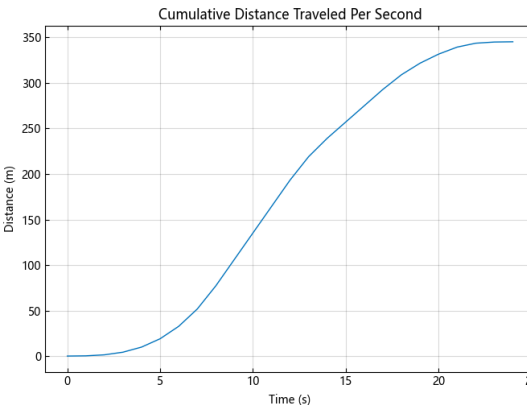
函数名称	功能
integral	函数表达式数值积分
integral2	二重积分
integral3	三重积分
quadgk	高斯-勒让德算法
cumtrapz	累计梯形数值积分
trapz	梯形数值积分

示例1: 如何对一组离散速度数据进行数值积分以逼近行驶距离。:

```
vel = [0 0.45 1.79 4.02 7.15  
11.18 16.09 21.90 29.05 29.05  
29.05 29.05 22.42 17.9 17.9  
17.9 14.34 11.01 8.9 6.54  
2.03 0.55 0];  
time = 0:24;  
figure()  
plot(time, vel, "-*")  
grid("on")  
title("Automobile Velocity")  
xlabel("Time (s)")  
ylabel("Velocity (m/s)")  
xverts = [time[1:end-1]';  
time[1:end-1]'; time[2:end]'];  
time[2:end]'];  
yverts = [zeros(1, 24);  
vel[1:end-1]'; vel[2:end]';  
zeros(1, 24)];  
p = patch(xverts, yverts, "b",  
linewidth=1.5);  
distance = trapz(vel);  
cdistance = cumtrapz(vel);  
figure()  
plot(cdistance)  
title("Cumulative Distance  
Traveled Per Second")  
xlabel("Time (s)")  
ylabel("Distance (m)")
```



trapz使用数据点进行离散积分以创建梯形非常适合处理不连续的数据集



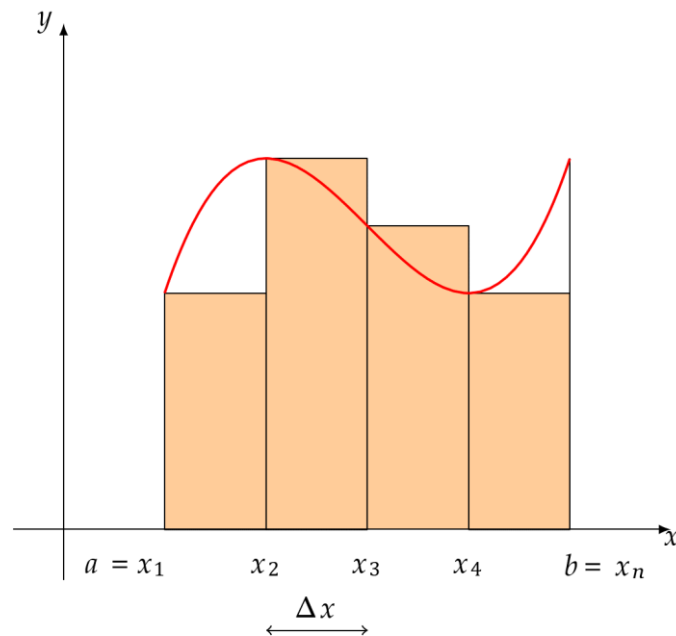
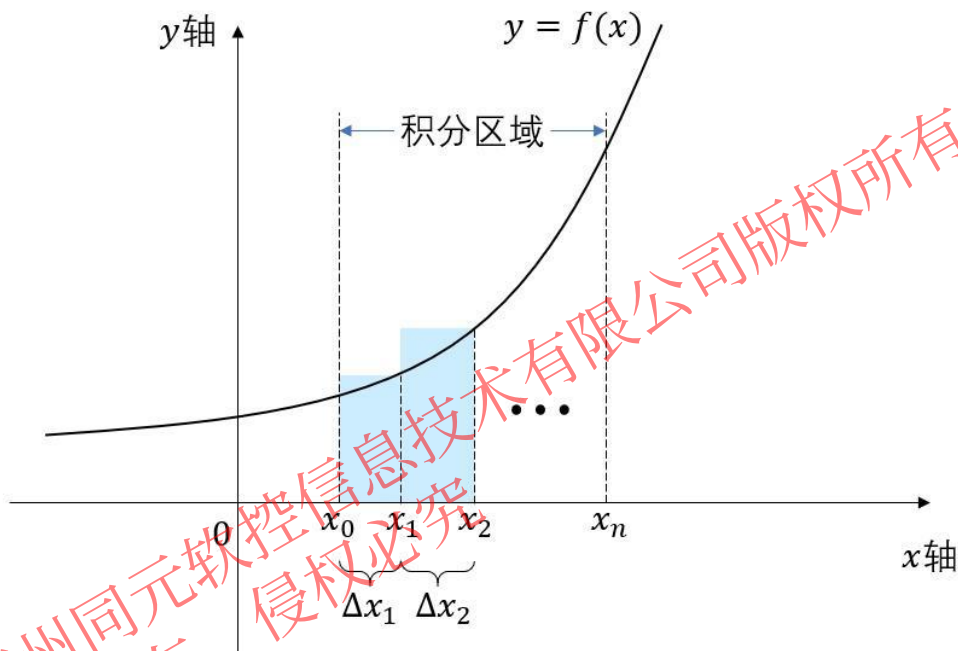
cumtrapz函数与trapz密切相关。trapz仅返回最终的积分值，而cumtrapz还在向量中返回中间值。



4.1 数值积分

在数学分析中，许多定积分不能用已知的积分公式得到精确值，利用给定函数的定积分计算不总是可行的，数值积分是计算定积分数值的方法和理论，用于求定积分的近似值。

求解定积分的数值方法多种多样，如简单的梯形法、辛普生(Simpson)法、牛顿-柯特斯(Newton-Cotes)法等都是经常采用的方法。它们的基本思想都是将整个积分区间 $[a, b]$ 分成 n 个子区间 $[x_i, x_{i+1}]$, $i = 1, 2, \dots, n$, 其中 $x_1 = a$, $x_{n+1} = b$ 。这样求定积分问题就分解为求和问题。



4.1.1 数值积分-一重积分

以Syslab中的一重积分函数为例：quad、quadl、quadgk、integral

`q, e = quad(fun, a, b)`

- simpson公式，适用于精确度较低，被积函数平滑性较差的数值积分

`Q, fcnt = quadl(fun, a, b, tol, trace)`

- Lobatto公式，精确度要求较高，被积函数也更为平滑的数值积分，fcnt为函数计算次数

`quadgk(f, a, b, c...; rtol=sqrt(eps), atol=0, maxevals=107, order=7, norm=norm)`

- Gauss-Kronrod数值积分，函数的精确度最高，会产生震荡被积函数。它支持无限区间并且可以处理端点处的适度奇异性。它还支持沿分段线性路径的围道积分

`q, e = integral(fun, xmin, xmax; Name=Value)`

- 使用全局自适应积分和默认误差容限在xmin至xmax间以数值形式为函数fun求积分。指定具有一个或多个Name=Value关键字参数的其他选项。例如，指定rtol，后跟实数，可调整相对误差

4.1.2 数值积分-二重积分

Syslab中的二重积分函数: quad2d、dblquad、integral2

$q, E = \text{quad2d}(\text{fun}, a, b, c, d; \text{Key}=\text{Value})$

- 逼近 $\text{fun}(x, y)$ 在平面区域 $a \leq x \leq b$ 和 $c(x) \leq y \leq d(x)$ 上的积分。边界 c 和 d 均可为标量或函数句柄并返回绝对误差 $E = |q - I|$ 的逼近上限, 其中 I 是积分的确切值。指定具有一个或多个 $\text{Key}=\text{Value}$ 关键字参数的其他选项。例如, 您可以指定 AbsTol 和 RelTol 来调整算法必须满足的误差阈值

$q = \text{dblquad}(\text{fun}, \text{xmin}, \text{xmax}, \text{ymin}, \text{ymax}; \text{tol}=1\text{e-}6, \text{quadf1}=\text{ty_quad})$

- 调用 quad 函数来计算 $\text{xmin} \leq x \leq \text{xmax}$, $\text{ymin} \leq y \leq \text{ymax}$ 矩形区域上的二重积分 $\text{fun}(x, y)$, 输入参数 fun 是一个函数句柄, 它接受标量 x , 标量 y , 并返回被积函数值, 使用容差 tol 代替默认值 $1.0\text{e-}6$, 使用指定为 quadf1 的求积法函数代替默认值 quad 。 quadf1 的有效值为 ty_quad 或用户指定的求积法的函数句柄, 该句柄与 quad 和 quadl 具有相同的调用顺序

$q, e = \text{integral2}(\text{fun}, \text{xmin}, \text{xmax}, \text{ymin}, \text{ymax}; \text{Name}=\text{Value})$

- 在平面区域 $\text{xmin} \leq x \leq \text{xmax}$ 和 $\text{ymin}(x) \leq y \leq \text{ymax}(x)$ 上逼近函数 $z = \text{fun}(x, y)$ 的积分。指定具有一个或多个 $\text{Name}=\text{Value}$ 关键字参数的其他选项

4.1.2 数值积分-二重积分

例：求二重定积分 $I = \int_{-1}^1 \int_{-2}^2 e^{-x^2/2} \sin(x^2 + y) dx dy$

#函数定义

```
Julia> fh(x,y)=exp(-x.^2/2).*sin(x.^2+y)
```

```
Julia> I1=quad2d(fh, -2,2, -1,1)  
(1.5744981414682055, 1.086590179084166e-6)
```

```
Julia> I2= dblquad(fh, -2,2, -1,1; tol=1e-10,quadf1=quadl)  
1.5744981592173604
```

```
Julia> I3=integral2(fh, -2,2, -1,1; atol=1e-8,rtol=1e-12)  
(0.8488435998870193, 9.93533839161782e-9)
```

4.1.3 数值积分-三重积分

Syslab中的三重积分函数: quad3d、triplequad、integral3

$q, e = \text{quad3d}(\text{fun}, x_{\min}, x_{\max}, y_{\min}, y_{\max}, z_{\min}, z_{\max}, \text{Name}=\text{Value})$

- 在区域 $x_{\min} \leq x \leq x_{\max}$ 、 $y_{\min}(x) \leq y \leq y_{\max}(x)$ 和 $z_{\min}(x, y) \leq z \leq z_{\max}(x, y)$ 逼近函数 $z = \text{fun}(x, y, z)$ 的积分, 指定具有一个或多个 $\text{Name}=\text{Value}$ 关键字参数的其他选项。

$q = \text{triplequad}(\text{fun}, x_{\min}, x_{\max}, y_{\min}, y_{\max}, z_{\min}, z_{\max}, \text{tol}, \text{method})$

- 对三维矩形区域 $x_{\min} \leq x \leq x_{\max}$ 、 $y_{\min} \leq y \leq y_{\max}$ 、 $z_{\min} \leq z \leq z_{\max}$ 求三重积分 $\text{fun}(x, y, z)$, 输入参数 fun 是一个函数句柄, 它接受标量 x , 标量 y , 标量 z 并返回被积函数值, 使用容差 tol 代替默认值 $1.0e-6$, 使用指定为 method 的求积法函数代替默认值 quad 。

$q, e = \text{integral3}(\text{fun}, x_{\min}, x_{\max}, y_{\min}, y_{\max}, z_{\min}, z_{\max}, \text{Name}=\text{Value})$

- 在区域 $x_{\min} \leq x \leq x_{\max}$ 、 $y_{\min}(x) \leq y \leq y_{\max}(x)$ 和 $z_{\min}(x, y) \leq z \leq z_{\max}(x, y)$ 逼近函数 $z = \text{fun}(x, y, z)$ 的积分, 指定具有一个或多个 $\text{Name}=\text{Value}$ 关键字参数的其他选项。

4.1.3 数值积分-三重积分

例：用数值方程求解三重积分 $I = \int_0^1 \int_0^\pi \int_0^\pi 4xze^{-x^2y-z^2} dz dy dx$

#函数定义

```
Julia> fh(x,y,z)=4*x.*z.*exp(-(x.^2).*y-z.^2)
```

#使用quad3d函数对三重积分进行数值计算

```
Julia> q,e = quad3d(fh,0,1,0,pi, 0,pi,atol=1e-12)  
(2.5356917741148663, 9.999969847071944e-13)
```

#使用triplequad函数对三重积分进行数值计算，并设置积分法为quadl

```
Julia> q = triplequad(fh,0,1,0,pi, 0,pi,1e-10, quadl)  
1.7327622230309039
```

#使用integral3函数对三重积分进行数值计算

```
Julia> q,e=integral3(fh,0,1,0,pi,0,pi,atol=1e-8,rtol=1e-12)  
(2.535691774124124, 9.99789315994994e-9)
```

#指定积分法

```
Julia> q,e=integral3(fh,0,1,0,pi,0,pi,atol=1e-8,rtol=1e-  
12,method=:cuhre)  
(2.5356917791362537, 9.25601013761248e-9)
```

4.1.4 数值积分-向量化积分

Syslab中的向量化积分函数: `quadv`、`integral`

`Q,fcnt = quadv(fun,a,b,tol,trace)`

- 使用递归自适应Simpson积分法求取复数数组值函数fun从a到b的近似积分，误差小于1.e-6并返回函数计算数。fun是函数句柄。函数Y=fun(x)应接受标量参数x并返回数组结果Y，其分量是在x处计算的被积函数。范围a和b必须是有限的,对所有积分使用绝对误差容限tol，代替默认值1.e-6,trace指定为true时在递归期间显示[fcnt a b-a Q[1]]的值。

`q,e = integral(fun, xmin,xmax;Name = Value)`

- 使用全局自适应积分和默认误差容限在xmin至xmax间以数值形式为函数fun求积分。指定具有一个或多个Name=Value关键字参数的其他选项。例如，指定 `rtol`，后跟实数，可调整相对误差。

例：求函数 $f(x) = \frac{1}{n+x}$ 在n取1~10情况下，x从0~1的积分

#函数定义

```
Julia> fh= (x,n) -> 1 ./((1:n).+x)
```

```
Julia> Q,fcnt = quadv(x-> fh(x,10),0,1)
```

```
([0.69314719986297, 0.4054651083775654, 0.28768207247245975, 0.22314355131746128, 0.1823215567947161,  
0.15415067982748842, 0.1335313926246057, 0.11778303565641765, 0.10536051565784188, 0.09531017980433254], 17)
```

```
Julia> q,e=integral.(x-> fh(x,10),0,1)
```

```
([0.6931471805599453, 0.4054651081081644, 0.2876820724517809, 0.22314355131420976, 0.18232155679395462,  
0.1541506798272583, 0.13353139262452263, 0.11778303565638346, 0.10536051565782631, 0.09531017980432487],  
1.99318352275991e-11)
```

4.1.5 数值积分-离散数据积分

$Q = \text{trapz}(X,Y,\text{dim})$: 梯形数值积分, 通过已知参数 X,Y 按 dim 维使用梯形公式进行积分。

例1:

```
Julia> x=0:pi/100:pi/2;  
Julia> y=sin.(x);  
Julia> Q = trapz(x,y)  
0.968509576339474
```

例2:

#积分区间定义

```
Julia> x=-3:0.01:3;
```

```
Julia> y=-5:0.01:5;
```

#生成网格

```
Julia> X,Y=meshgrid2(x,y);
```

```
Julia> F=X.^2+Y.^2;
```

```
Julia> surf(X,Y,F)
```

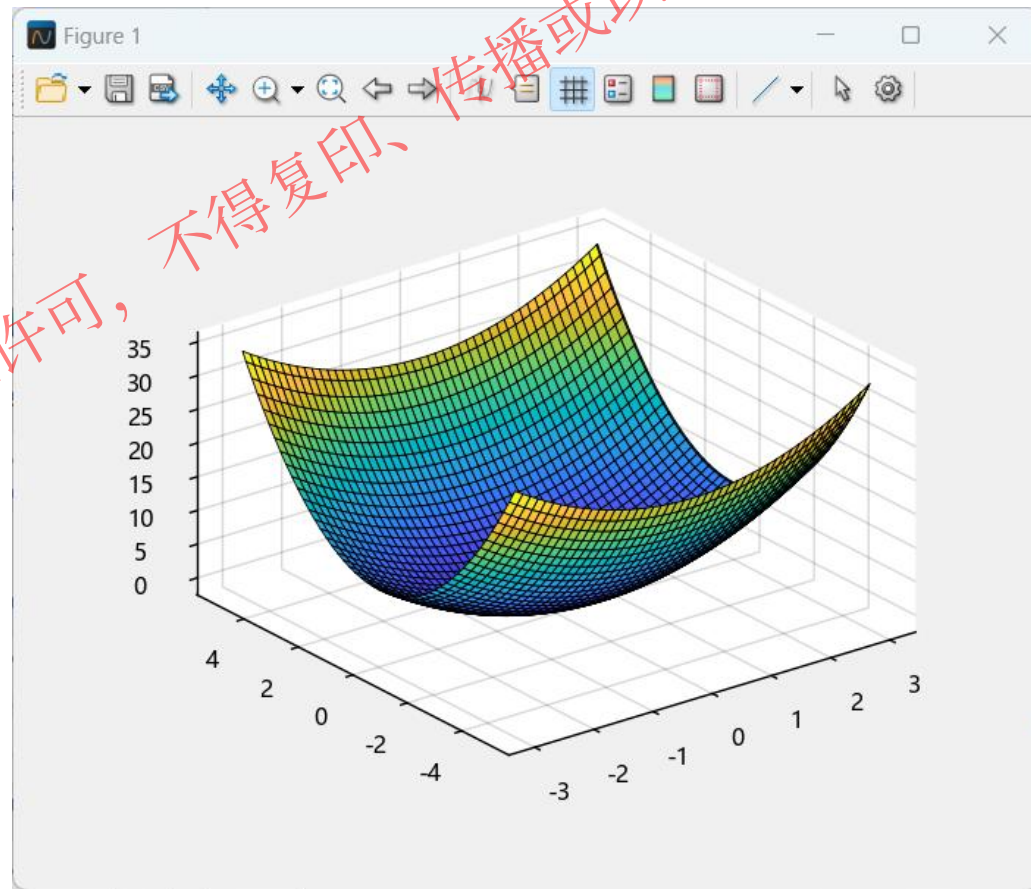
#使用trapz先对x求积分以生成列向量。然后, y上的积分可将列向量减少为单个标量

```
Julia> Q=trapz(y,trapz(x,F,2))
```

```
1×1 adjoint(::Vector{Float64}) with eltype  
Float64:
```

```
680.002
```

#trapz 稍微高估计确切答案680, 因为 $f(x,y)$ 是向上凹的



4.2 数值积分与微分方程

- 数值积分类板块函数提供无论函数表达式是否已知都可以求积分的近似值的解决方案
- **微分方程**板块函数提供ODE(常微分方程)、DDE(时滞微分方程)、BVP(边界值问题)、SDE(随机微分方程)等微分方程求解的解决方案

二. 微分方程

模型创建方法

函数名称	功能
ODEProblem	常微分方程问题创建
DDEProblem	时滞微分方程模型创建
BVPProblem	边界值问题模型创建
SDEProblem	随机微分方程模型创建
DAEProblem	微分代数方程模型创建

微分方程分类

ODE				
ODE	DAE	SDE	DDE	Discrete Problem
• ODE • 2nd Order ODE • Dynamical ODE • Steady State • RODE • Stiff Equations • Non-Stiff Equations • SODE • Explicit ODE • Implicit ODE		• SDAE • SDDE	• DDE • DDDE	

4.2 数值微分

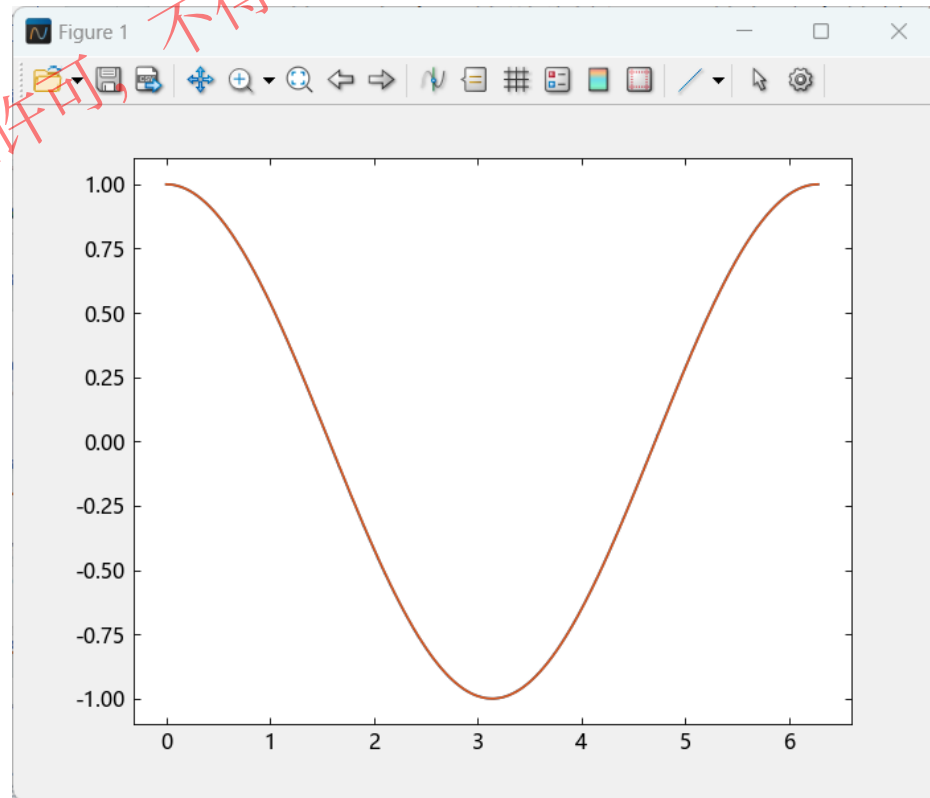
函数 $f(x)$ 在点 x_0 的微分接近于函数在该点的差分，而 f 在点 x 的导数接近于函数在该点的差商

Syslab中提供了求向前差分的函数diff，调用格式如下：

- $D=\text{diff}(x)$:计算向量 x 的向前差分， $dx(i)=x(i+1)-x(i), i=1,2,\dots,n-1$ 。
- $D=\text{diff}(x,n)$:计算向量 x 的 n 阶向前差分。例如， $\text{diff}(x,2)=\text{diff}(\text{diff}(x))$ 。
- $D=\text{diff}(x,n,\text{dim})$:计算矩阵 A 的 n 阶差分， $\text{dim}=1$ 时（默认状态），按行计算差分， $\text{dim}=2$ 时，按列来计算差分

例：设 $f(x) = \sin(x)$ ，在 $[0,2\pi]$ 范围内随机采样，计算 $f'(x)$ 的近似值，并与理论值 $f'(x) = \cos(x)$ 进行比较

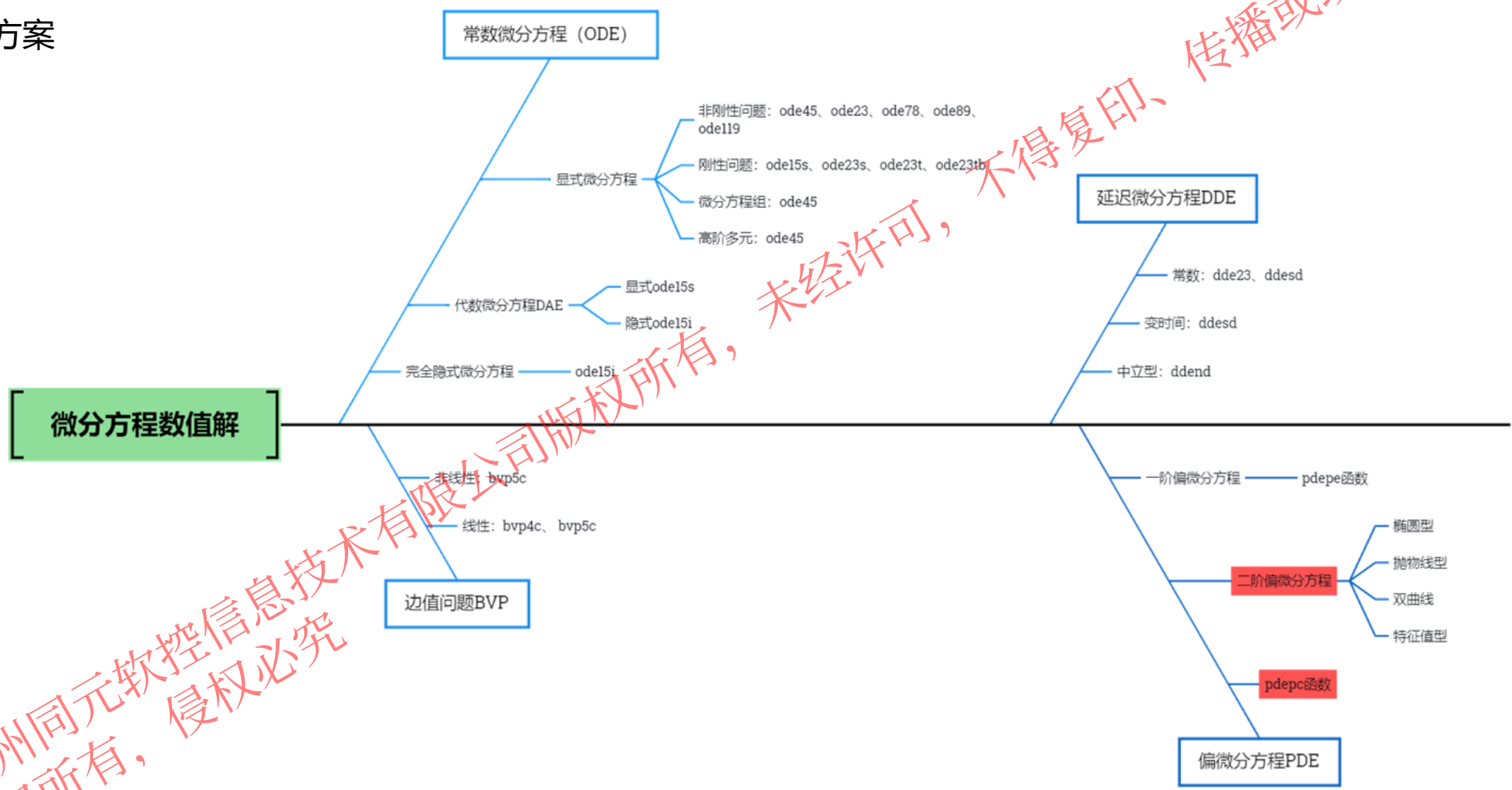
```
#随机采样
x=[0;sort(2*pi*rand(5000));2*pi]
y=sin.(x)
#求f在x处的导数值
f1=diff(y)./diff(x)
f2=cos.(x[1:end-1])
#两个解的曲线绘制
plot(x[1:end-1],f1,x[1:end-1],f2)
#近似值解与理论值的比较
d=norm(f1-f2)
#ans=0.044821948319216105
```



4.2 微分方程

微分方程是指含有未知函数及其导数的关系式。

微分方程板块函数提供ODE(常微分方程)、DDE(时滞微分方程)、BVP(边界值问题)、PDE（偏微分方程）等微分方程求解的解决方案



4.2.1 常微分方程的数值求解

□ 常微分方程求解的一般概念

常微分方程

常微分方程 (ODE) 包含与一个自变量 t (通常称为时间) 相关的因变量 y 的一个或多个导数。此处用于表示 y 关于 t 的导数的表示法对于一阶导数为 y' , 对于二阶导数为 y'' , 依此类推。ODE 的阶数等于 y 在方程中出现的最高阶导数。

例如, 这是一个二阶 ODE:

$$y'' = 9y$$

求解常微分方程初值问题就是寻找函数 $y(t)$ 使之满足如下方程:

$$y' = f(t, y), t_0 < t < b$$

$$y(t_0) = y_0$$

所谓其数值解法, 就是求 $y(t)$ 在离散结点 t_n 处的函数近似值 y_n 的方法, $y_n \sim y(t_n)$

这些近似值称为常微分方程初值问题的数值解。相邻两个结点之间的距离称为步长。

4.2.1 常微分方程的数值求解

□ 常微分方程求解的一般概念

- 单步法: 在计算 y_{n+1} 时只用到前一步的 y_n , 因此在有了初值之后, 就可以逐步往下计算。其代表是龙格-库塔 (Runge-Kutta) 法。
- 多步法: 在计算 y_{n+1} 时,除了用到前一步的值 y_n 之外,还要用到 y_{n-p} ($p = 1, 2, \dots, k, k > 0$) 的值, 即前面的 k 步。其代表就是亚当斯(Adams)法。

在每一步, 求解器都对之前各步的结果应用一个特定算法。在第一个这样的时间步, 初始条件将提供继续积分所需的必要信息。

最终结果是, ODE 求解器返回一个时间步向量 $t = [t_0, t_1, t_2, \dots, t_f]$ 以及在每一步对应解 $y = [y_0, y_1, y_2, \dots, y_f]$ 。

4.2.1 常微分方程的数值求解

□ 常微分方程求解的一般概念

Syslab提供了多个求常微分方程初值问题数值解的函数，一般调用格式为：

$$t, y = \text{solver}(\text{filename}, \text{tspan}, y0, \text{option})$$

其中， t 和 y 分别给出时间向量和相应的数值解。

solver 为求常微分方程数值解的函数。

filename 是定义 $f(t, y)$ 的函数名，该函数必须返回一个列向量。

tspan 形式为 $[t0, tf]$ ，表示求解区间。

$y0$ 是初始状态向量。

Option 是可选参数用于设置求解属性，常用的属性包括：相对误差值 RelTol (默认值是 10^{-3})和绝对误差值 AbsTol (默认值是 10^{-6})。

4.2.1 常微分方程的数值求解

□ 常微分方程求解的一般概念

非刚性求解器

函数名	采用方法	简介
ode45	4阶或者5阶的Runge-Kutta算法，中精度	求解非刚性微分方程 - 中阶方法
ode23	2阶或者3阶的Runge-Kutta算法，低精度	求解非刚性微分方程 - 低阶方法
ode78	7阶或者8阶的Runge-Kutta算法，高精度	求解非刚性微分方程 - 高阶方法
ode89	8阶或者9阶的Runge-Kutta算法，高精度	求解非刚性微分方程 - 高阶方法
ode113	Adams算法，精度可到 $10^{-3} \sim 10^{-6}$ ，时间比ode45短	求解非刚性微分方程 - 变阶方法

刚性求解器

函数名	采用方法	简介
ode15s	Gear 's反向数值微分算法，中精度	求解刚性微分方程和 DAE - 变阶方法
ode23s	2阶Rosebrock算法，低精度，时间比ode15s短	求解刚性微分方程 - 低阶方法
ode23t	梯形算法	求解中等刚性的 ODE 和 DAE - 梯形法则
ode23tb	梯形算法，低精度，时间比ode15s短	求解刚性微分方程 - 梯形法则 + 后向差分公式

完全隐式求解器

函数名	简介
ode15i	解算全隐式微分方程 - 变阶方法
decic	为 ode15i 计算一致的初始条件

4.2.2 常微分方程的数值求解-显式微分方程

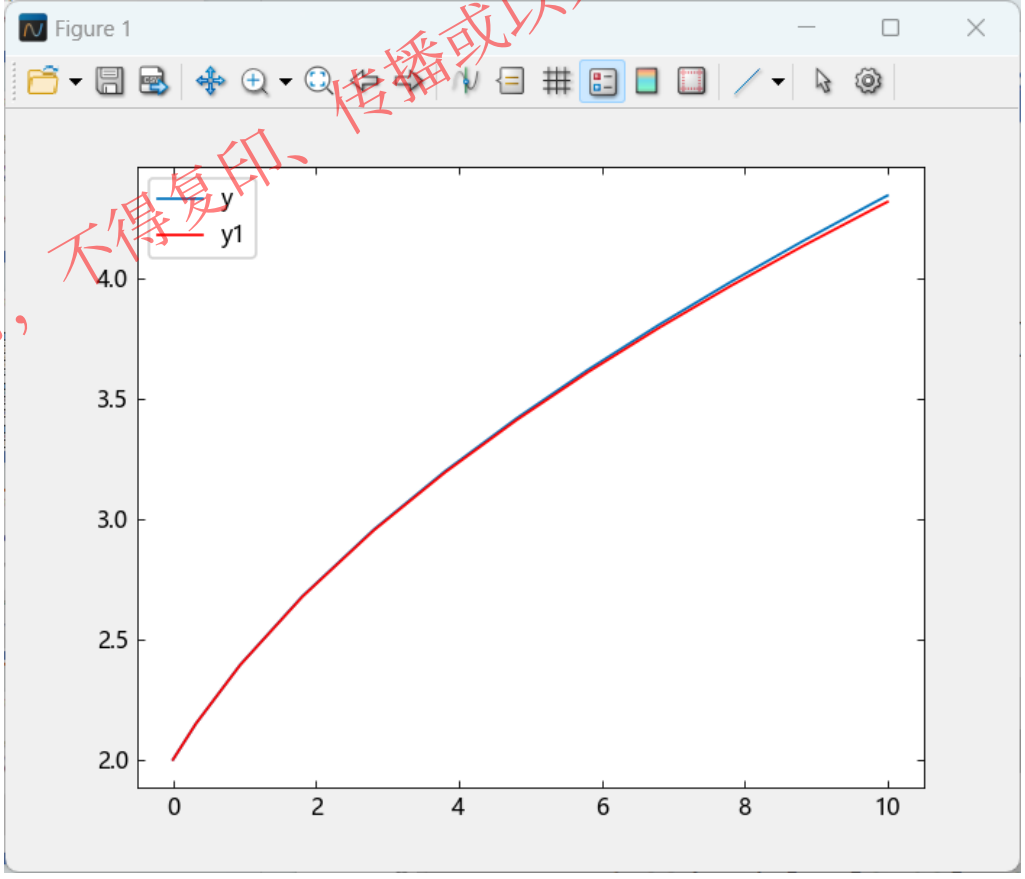
显式微分方程-非刚性问题

➤ 示例1:

求解微分方程的初值问题，并与精确解 $y(t) = \sqrt{t+1} + 1$ 比较

$$y' = \frac{y^2 - t - 2}{4(t + 1)}, \quad 0 \leq t \leq 10$$
$$y(0)=2$$

```
function model(t, y)
    dydt = (y.^2 - t - 2) ./ (4 .* (t + 1))
    return dydt
end
t, y, = ode23(model, [0 10], 2)
y1 = sqrt.(t + 1) .+ 1
plot(t, y, "-", t, y1, "r")
legend("y", "y1")
```



4.2.2 常微分方程的数值求解-显式微分方程

□ 显式微分方程-刚性问题

刚性问题

有一类常微分方程，其解的分量有的变化很快，有的变化很慢，且相差悬殊，这就是所谓的刚性问题(Stiff)。

对于刚性问题，数值解算法必须取很小步长才能获得满意的结果，导致计算量会大大增加。解决刚性问题需要有专门方法。非刚性算法可以求解刚性问题，只不过需要很长的计算时间。

➤ 示例2:

$$\begin{aligned} y' &= y^2 - y^3 \\ y(0) &= \lambda \\ 0 \leq t &\leq \frac{2}{\lambda} \end{aligned}$$

其中y是t的函数

```
function model2(t, y)
    dydt = y.^2 - y.^3
    return dydt
end

lamda=0.01
```



4.2.2 常微分方程的数值求解-显式微分方程

```
lamda=0.01
```

```
t, y, = ode45(model2, [0 2/lamda], lamda)
@time t, y, = ode45(model2, [0 2/lamda], lamda)
# 0.000298 seconds (4.42 k allocations: 261.641 KiB)
@show length(t)
#157
```

运行时间: 0.000298s

点数: 157

结果表明: 这时这个方程不算很刚性

```
lamda=1e-5
```

```
t, y, = ode45(model2, [0 2/lamda], lamda)
@time t, y, = ode45(model2, [0 2/lamda], lamda)
# 0.506465 seconds (3.52 M allocations: 759.346 MiB,
26.02% gc time)
@show length(t)
#120565
```

运行时间: 0.506465s

运行点数: 120565

这次时间明显加长, 计算的点数激增

结果表明: 此时这个方程表现为刚性

@time用于执行表达式的宏, 打印执行所需的时间、分配数及其字节总数

@show显示表达式和结果, 返回结果

4.2.2 常微分方程的数值求解-显式微分方程

```
lamda=1e-5

t, y, = ode23s(model2, [0 2/lamda], lamda)
@time t, y, = ode23s(model2, [0 2/lamda], lamda)
# 0.001406 seconds (24.66 k allocations: 1.372 MiB)

@show length(t)
#98
```

运行时间: 0.001406s

运行点数: 98

对于刚性问题，我们需要改变求解算法。我们选择以“s”结尾的函数，他们专门用于求解刚性方程。

计算时间大大缩短，计算的点数大大减少。

4.2.3 常微分方程的数值求解-完全隐式微分方程

完全隐式常微分方程

完全隐式常微分方程的形式为： $f(t, y, y') = 0$ 。

用法为：`t,y = ode15i(odefun, tspan, y0, yp0, options)`

- odefun：为待求解方程
- tspan：用于指定积分区间
- y0,yp0：分别用于指定初值有 $y(t_0)$ 和 $y'(t_0)$ ， $f(t_0, y_0, yp_0) = 0$ 。
- options：可选参数，用于指定积分方法

`y0mod,yp0mod = decic(odefun,t0,y0,fixed_y0,yp0,fixed_yp0)`：为 ode15i 计算一致的初始条件。
给定y0，fixed_y0取1，否则为0，给定yp0，则fixed_yp0取1，否则取0。



4.2.3 常微分方程的数值求解-完全隐式微分方程

完全隐式微分方程

示例:

$$\begin{cases} ty^2(y')^3 - y^3(y')^2 + t(t^2 + 1)y' - t^2y = 0 \\ y(1) = \sqrt{3/2} \longrightarrow y'(1) = ? \end{cases}$$

编写方程代码:

```
function weissinger(t, y, yp)
    return @. t * y^2 * yp^3 - y^3 * yp^2
    + t * (t^2 + 1) * yp - t^2 * y
end
```

计算一致的初始条件: 使用 decic 加上ODE15i返回的一致初始条件, 在时间区间 [1 10] 上求解 ODE。

```
t0 = 1
y0 = sqrt(3 / 2)
yp0 = 0
y0, yp0 = decic(weissinger, t0, y0, 1, yp0, 0)
t,y = ode15i(weissinger,[1 10],y0,yp0)
```



4.2.3 常微分方程的数值求解-完全隐式微分方程

完全隐式微分方程

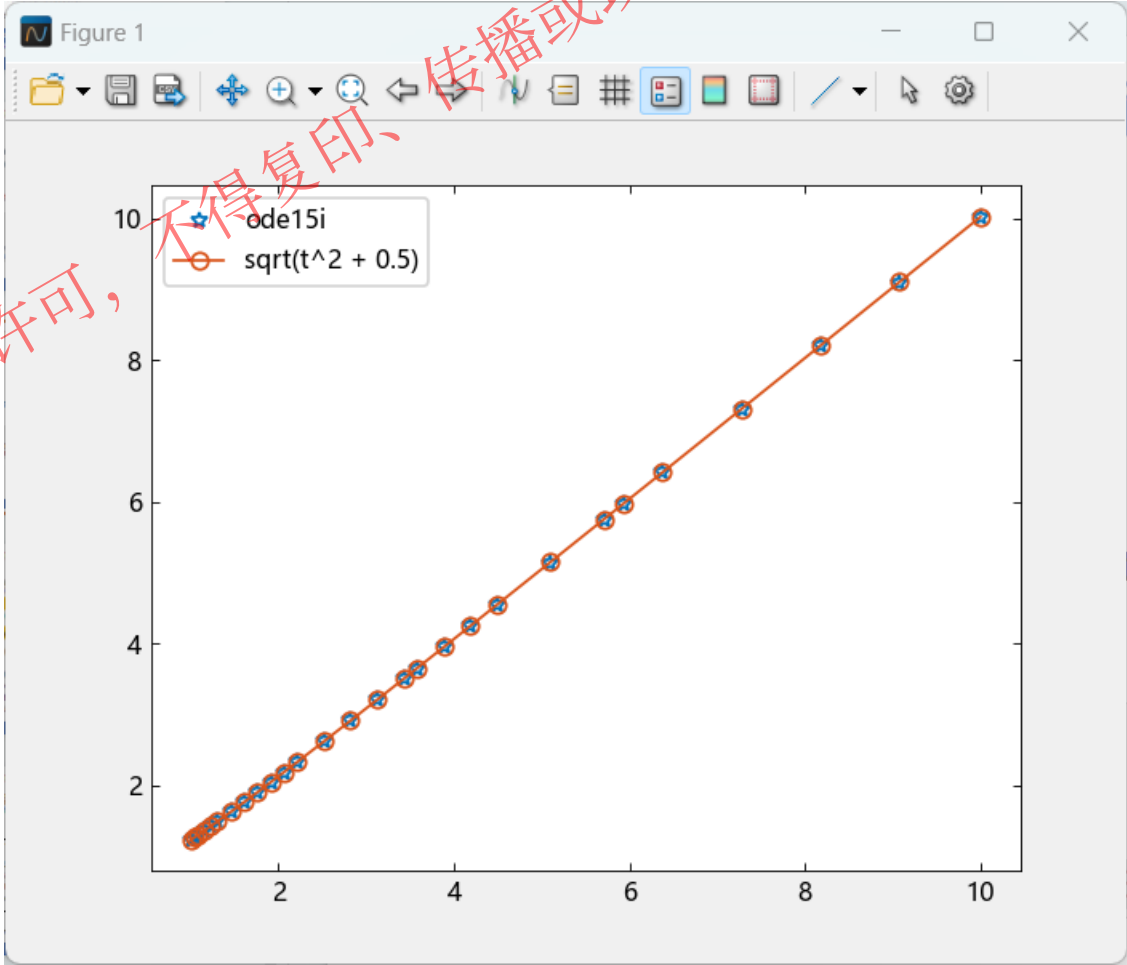
绘制结果

此 ODE 的精确解为

$$y(t) = \sqrt{t^2 + \frac{1}{2}}$$

绘制 ode15i 计算的数值解 y 对解析解 ytrue 的图。

```
#解析解
ytrue = @. sqrt(t^2 + 0.5)
plot(t, y, "*", t, ytrue, "-o")
legend(["ode15i", "sqrt(t^2 + 0.5)"])
```



4.2.4 常微分方程的数值求解-代数微分方程

代数微分方程-DAEs

微分方程的形式是多种多样，一般来说，很多很多高阶微分方程可以通过变量替换转化成一阶微分方程组，即可以写成下面的形式①：

$$M(t, y)y' = F(t, y)$$

$M(t, y)$ 称为质量矩阵，如果其非奇异的话，上式可以写成式子②：

$$y' = M(t, y)^{-1}F(t, y)$$

将②式右半部分用odefun表示出来，就是ode45，ode23等常微分方程初值问题求解的输入参数odefun。

如果质量矩阵奇异的话，①式称为微分代数方程组 (differential algebraic equations, DAEs.)

可以利用求解刚性微分方程的函数如ode15s、ode23s等来求解

从输入形式上看，求解DAEs和求解普通的ODE很类似，主要区别是需要给**微分方程求解器指定质量矩阵**。

隐式微分方程无法写成①或者②的形式

4.2.4 常微分方程的数值求解-代数微分方程

所有 ODE 求解器都可以解算 $y' = f(t, y)$ 形式的方程组，或涉及质量矩阵 $M(t, y)y' = f(t, y)$ 的问题。

ode23s 求解器只能解算质量矩阵为常量的问题。

ode15s 和 **ode23t** 可以解算具有奇异质量矩阵的问题，使用 odeset 的 Mass 选项指定质量矩阵。

ode15s的求解：

t, y = ode15s(odefun, tspan, y0, options)

-odefun：为待求解方程

-tspan：用于指定积分区间

-y0：分别用于指定初值。

-options：使用 odeset 函数创建的参数，定义的积分设置

Tips: options = odeset(Name, Value, ...) 创建 options 结构体，您可以将其作为参数传递给 ODE 和 PDE 求解器

4.2.4 常微分方程的数值求解-代数微分方程

求解代数微分方程示例：

$$\begin{cases} y_1' = -0.2y_1 + y_2y_3 + 0.3y_1y_2 \\ y_2' = 2y_1y_2 - 5y_2y_3 - 2y_2^2 \\ y_1 + y_2 + y_3 = 1 \end{cases}$$

微分方程

代数方程

初值： $y_1(0) = 0.8, y_2(0) = 0.1, y_3(0) = 0.1$

切换为矩阵形式的表示：

$$M(t,y)y' = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 0 \end{bmatrix} \begin{bmatrix} dy_1 / dt \\ dy_2 / dt \\ dy_3 / dt \end{bmatrix} = \begin{bmatrix} -0.2y_1 + y_2y_3 + 0.3y_1y_2 \\ 2y_1y_2 - 5y_2y_3 - 2y_2^2 \\ y_1 + y_2 + y_3 - 1 \end{bmatrix} = F(t,y)$$



4.2.4 常微分方程的数值求解-代数微分方程

- 定义待求解的方程

```
function test_ode15s(t,y)
    return [
        -0.2 * y[1] + y[2] .* y[3] + 0.3 * y[1] .* y[2]
        2 * y[1] .* y[2] - 5 * y[2] .* y[3] - 2 * y[2].^2
        y[1] + y[2] + y[3]-1
    ]
end
```

- 定义初始值向量y0
- 定义矩阵M
- 定义区间
- 定义结构体
- 调用ode15s函数
- 绘图

```
y0=[0.8,0.1,0.1]

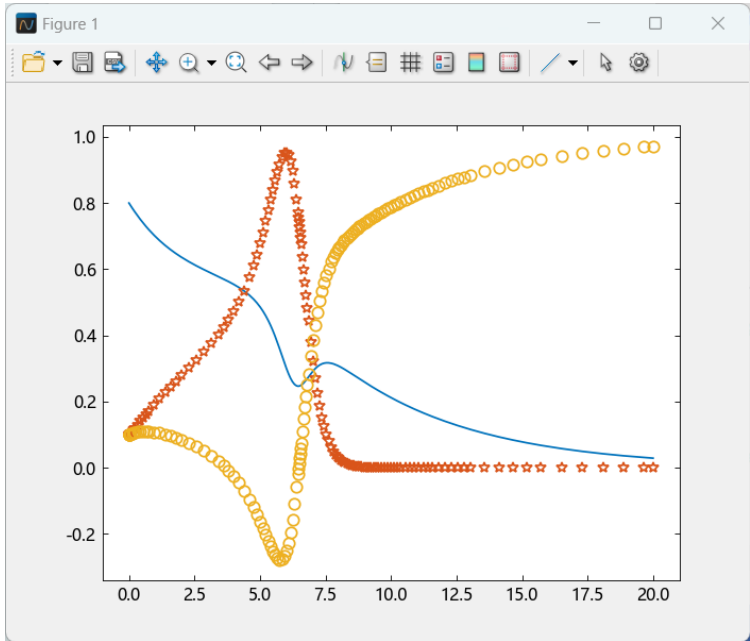
M=[1 0 0;0 1 0;0 0 0]

tspan = [0, 20]

options = odeset(mass=M, reltol=1e-4, abstol=[1e-6;
1e-6; 1e-6])

t, y, = ode15s(test_ode15s, tspan, y0, options);

plot(t, y[:, 1], "-", t, y[:, 2], "*", t, y[:, 3], "o")
```



4.2.5 微分方程的边界值求解

微分方程分为**初值问题**和**边值问题**。

- 初值问题是已知微分方程的初始条件，即自变量为零时的函数值，一般可以用欧拉法、龙格库塔法来求解。
- 边值问题则是已知微分方程的边界条件，即自变量在边界点时的函数值。

上节我们介绍的常微分方程，主要是微分方程的初值问题。本节介绍二阶常微分方程边值问题的建模与求解。

求解器	说明
bvp4c	求解边界值问题 - 四阶方法
bvp5c	求解边界值问题 - 五阶方法
bvpinit	得出边界值问题求解器的初始估计值

BVP 求解器的选择

软件提供了求解器 bvp4c 和 bvp5c 来求解 BVP。在大多数情况下，可以互换使用这些求解器。这些求解器之间的主要区别在于 bvp4c 实现四阶公式，而 bvp5c 实现五阶公式。

除两个求解器之间的误差容忍含义以外，bvp5c 函数的使用方法与 bvp4c 完全类似。如果 $S(x)$ 近似于解 $y(x)$ ，则 bvp4c 控制残差 $|S'(x) - f(x,S(x))|$ 。这种方法间接控制真误差 $|y(x) - S(x)|$ 。使用 bvp5c 直接控制真误差。

4.2.5 微分方程的边值问题

□ 边值问题

只含边界条件作为定解条件的常微分方程求解问题，称为常微分方程的边值问题（boundary value problem）。

- 若边界条件指定二点数值，称为狄利克雷边界条件（第一类边值条件），
- 此外也有指定二个特定点上导数的边界条件，称为诺伊曼边界条件（第二类边值条件）。
- 第三类边值条件：给出未知函数在边界上的函数值和外法线的方向导数的线性组合。

BVP 求解器 bvp4c 和 bvp5c 用于处理以下形式的 ODE 方程组：

$$y' = f(x, y)$$

其中， x 是自变量， y 是因变量， $y' = \frac{dy}{dx}$

形式：

```
sol = bvp4c(odefun,bcfun,solinit)
sol = bvp4c(odefun,bcfun,solinit,options)
sol = bvp5c(odefun,bcfun,solinit)
sol = bvp5c(odefun,bcfun,solinit,options)
```

Tips:

需要编写一个将方程表示为一阶方程组的函数、一个边界条件函数和一个初始估计值函数。然后 BVP 求解器使用这三个输入来求解方程



4.2.5 微分方程的边值问题

□ 边值问题

方程函数-odefun

示例：在syslab中求解一个二阶的BVP

$$y'' + y = 0$$

该方程在区间 $[0,\pi/2]$ 上进行定义，受限边界 $y(0) = 0$ 且 $y(\pi/2) = 2$

编写一个用于编写方程代码的函数。使用代换法将方程重写为一阶方程组。

$$y1 = y$$
$$y2 = y'$$

重写

$$y'1 = y2$$
$$y'2 = -y1$$

```
function bvpfun(x, y)
    return [y[2]; -y[1]]
end
```

或者

```
function bvpfun(x, y) # equation being solved
    yp=zeros(2,1);
    yp[1] = y[2]
    yp[2] =-y[1]
    return [y[2]; -y[1]]
end
```



4.2.5 微分方程的边值问题

□ 边值问题

边界条件-bcfun

在两点 BVP 的最简单情形中，ODE 的解在区间 $[a, b]$ 中求得，并且必须满足边界条件

$$g(y(a), y(b)) = 0.$$

要指定给定 BVP 的边界条件：

- 编写 $res = bcfun(ya, yb)$ 形式的函数，如果涉及未知参数，则使用 $res = bcfun(ya, yb, p)$ 形式。

将此函数作为第二个输入参数提供给求解器。该函数返回 res ，这是在边界点处的解的残差值。

例如，如果 $y(a) = 1$ 且 $y(b) = 2$ ，则边界条件函数为：

```
function bcfun(ya, yb)
    return [ya[1]; yb[1] - 2]
end
```


4.2.5 微分方程的边值问题

□ 边值问题

初始估计值-solinit

求解 BVP 过程的**重要部分**是提供所需解的估计值。

得出 BVP 问题的解的良好初始估计值可能是求解问题**最困难**的部分。

解的初始估计值

与初始值问题不同，边界值问题的解可以是：

- 无解
- 有限个解
- 无限多个解

常规指导原则

- 确保初始估计值满足边界条件，因为解也需要满足这些边界条件有限个解。
- 尝试将关于实际问题或预期解的信息尽可能多地纳入初始估计值。
- 考虑网格点的位置（解的初始估计值的 x 坐标）。
- 如果在较小区间内有一个已知的、更简单的解，则将它用作较大区间内的初始估计值。

4.2.5 微分方程的边值问题

□ 边值问题

初始估计值-solinit

bvpinit-得出边界值问题求解器的初始估计值函数

solinit = bvpinit(x,yinit) 使用初始网格 x 和初始解估计值 yinit 来得出边界值问题解的初始估计值。

```
solinit = bvpinit(x,yinit)
solinit = bvpinit(sol,[anew bnew])
solinit = bvpinit(___,parameters)
```

接上个示例：通过函数句柄提供初始估计值

合理地预期方程的解是振荡的，因此使用正弦和余弦函数可以很好地得出解及其固定边界点之间导数的初始估计值。

```
function guess(x) # initial guess for y and y'
    return [sin(x); cos(x)]
end
```

使用域 $[0, \pi/2]$ 中的 10 个等距网格点和初始估计值函数创建一个解结构体。

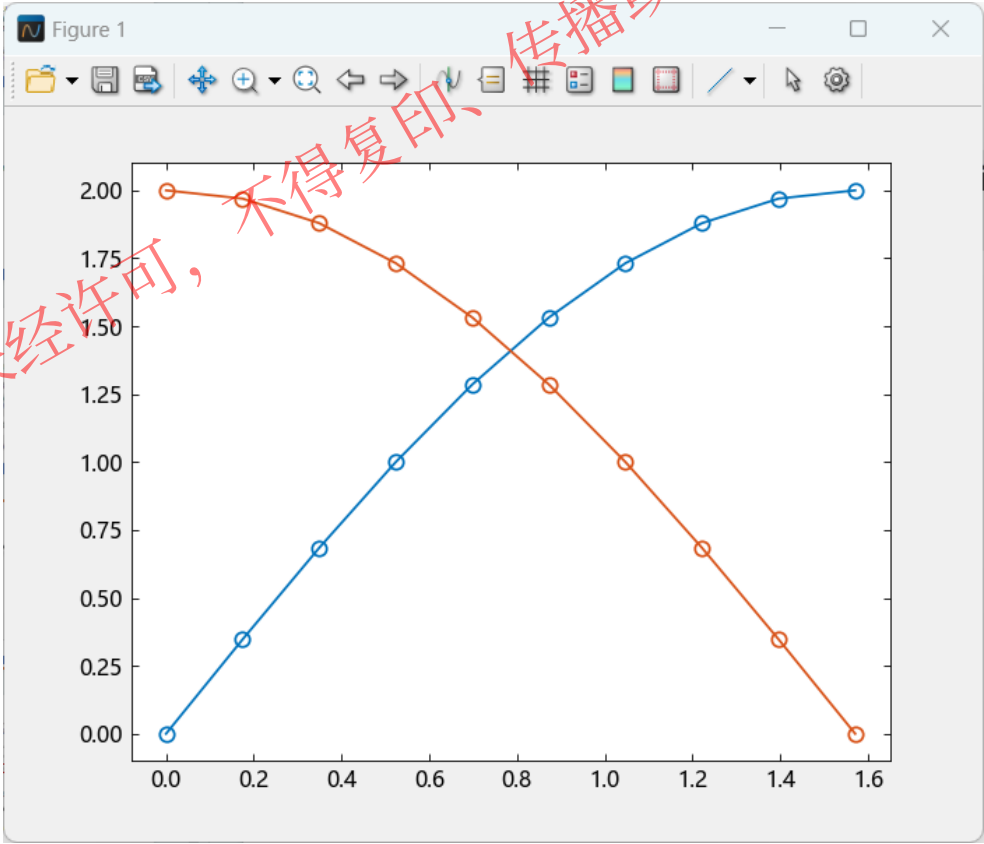
```
xmesh = LinRange(0, pi/2, 10)
solinit = bvpinit(xmesh, guess)
```

4.2.5 微分方程的边值问题

□ 边值问题

求解BVP-bvp4c

```
sol = bvp4c(bvpfun, bcfun, solinit);  
plot(sol.x, sol.y, "-o")
```



4.2.6 延迟（时滞）微分方程

- 在许多现实模型中，我们需要知道系统过去时刻的状态，这就形成了延迟微分方程（Delay Differential Equations）。
- 所谓延迟/时滞微分方程，指微分方程中信号不是同时发生的，除了当前时刻，还含有信号以前时刻的值。

典型（常数）	➡	$y' = f(t, y, y(t - t_1), y(t - t_2), \dots, y(t - t_n))$
变时间	➡	$y' = f(t, y, y(t - t_1), y(t - g(t)), \dots, y(t - t_n))$
中立型	➡	$y' = f(t, y, y(t - t_1), \dots, y(t - t_n), y'(t - t_{n+1}), \dots, y'(t - t_{n+m}))$

延迟（时滞）微分方程是微分方程表达式要依赖某些状态变量过去一些时刻

$$y' = f(t, y, y(t - t_1), y(t - t_2), \dots, y(t - t_n))$$

$t_1, t_2, \dots, t_n$ 是时间延迟项。既可以是常数也可以是关于 t 和 y 的函数。

当是常数时可以用dde23和ddesd来求解



4.2.6 延迟（时滞）微分方程

`sol = dde23(ddefun,lags,history,tspan,option)`

示例：求解延迟微分方程（常数）

$$y_1'(t) = y_1(t - 1)$$

$$y_2'(t) = y_1(t - 1) + y_2(t - 0.2)$$

$$y_3'(t) = y_2(t).$$

$t \leq 0 \quad y_1(t) = y_2(t) = y_3(t) = 1$ **history**

- ddefun表示函数句柄，形式dydt=ddefun(t,y,Z):
 - t 与当前t对应；y是对y(t)的近似；
 - Z(:,j)是对所有延迟为 t_j 的状态变量即 $y(t - t_j)$ 的估计， $t_j = lags(j)$
- lags存储各延迟常数的向量：比如 $t_1, t_2, \dots, t_n$
- history是描述 $t \leq t_0$ 时的状态变量的值的函数，可以为句柄形式或者常数形式。

	1	0.2	
	lags1	lags2	lags3
y1	Z(1,1)	Z(1,2)	Z(1,3)
y2	Z(2,1)	Z(2,2)	Z(2,3)
y3	Z(3,1)	Z(3,2)	Z(3,3)

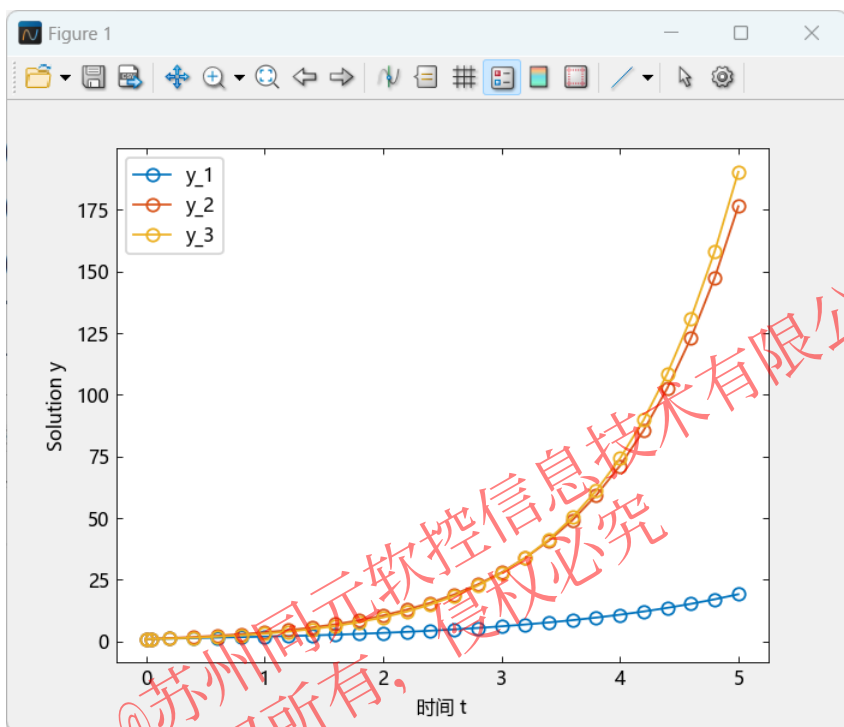
4.2.6 延迟（时滞）微分方程

$$y_1'(t) = y_1(t - 1)$$

$$y_2'(t) = y_1(t - 1) + y_2(t - 0.2)$$

$$y_3'(t) = y_2(t).$$

$$t \leq 0 \quad y_1(t) = y_2(t) = y_3(t) = 1$$



```
function ddex1de(t, y, Z)
    dydt = [ Z[1, 1],
             Z[1, 1]+Z[2, 2],
             y[2]]
    return dydt
end

lags = [1, 0.2]; #延迟常数向量
history = ones(3); #小于初值时的历史函数
tspan = [0, 5];
sol = dde23(ddex1de, [1, 0.2], history, [0, 5])
#sol = dde23(ddex1de, [1, 0.2], [1.0; 1.0; 1.0;;], [0, 5])
plot(sol.x, sol.y, "-o")
xlabel("时间 t");
ylabel("Solution y");
legend(["y_1", "y_2", "y_3"], loc="northwest");
```

4.2.7 一维偏微分方程PDE

在偏微分方程 (PDE) 中，要求解的函数取决于几个变量，微分方程可以包括关于每个变量的偏导数。偏微分方程可用于对波浪、热流、流体扩散和其他空间行为随时间变化的现象建模。

pdepe 要求解的一维方程大概可分为以下两类：

带时间导数的方程是抛物型方程。例如热方程 $\frac{\partial u}{\partial t} = \frac{\partial^2 u}{\partial x^2}$

不带时间导数的方程是椭圆型方程。例如拉普拉斯方程 $\frac{\partial^2 u}{\partial x^2} = 0$

求解一维 PDE

一维 PDE 包含函数 $u(x,t)$ ，该函数依赖于时间 t 和一个空间变量 x 。
PDE 求解器 pdepe 求解以下形式的一维抛物型和椭圆型 PDE 的方程组。

$$c\left(x, t, u, \frac{\partial u}{\partial x}\right) \frac{\partial u}{\partial t} = x^{-m} \frac{\partial}{\partial x} \left(x^m f\left(x, t, u, \frac{\partial u}{\partial x}\right) \right) + s\left(x, t, u, \frac{\partial u}{\partial x}\right)$$

- PDE 在 $t_0 \leq t \leq t_f$ 和 $a \leq x \leq b$ 时成立。
- 空间区间 $[a, b]$ 必须为有限值。
- m 可以是 0、1 或 2，分别对应平板、柱状或球面对称性。如果 $m > 0$ ，则 $a \geq 0$ 也必须成立。
- 系数 $f(x,t,u, \partial u/\partial x)$ 是通量项， $s(x,t,u, \partial u/\partial x)$ 是源项。
- 通量项必须取决于偏导数 $\partial u/\partial x$ 。



4.2.7 一维偏微分方程PDE

求解过程

要使用 pdepe 求解 PDE，您必须定义 c、f 和 s 的方程系数、初始条件、解在边界处的行为以及在其上计算解的点网格。

函数调用:

```
sol= pdepe(m,pdefun,icfun,bcfun,xmesh,tspan)
```

- m 是对称常量。
- pdefun 定义要求解的方程。
- icfun 定义初始条件。
- bcfun 定义边界条件。
- xmesh 是 x 的空间值向量。
- tspan 是 t 的时间值向量。
- xmesh 和 tspan 向量共同构成一个二维网格，pdepe 在该网格上计算解。

4.2.7 一维偏微分方程PDE

示例： $\frac{\partial u}{\partial t} = \frac{\partial^2 u}{\partial x^2}$

初始条件： $u(x,0)=1/2$

编写方程-pdefun

编写函数 pdefun, 它计算系数 c、f 和 s 的值。

使用函数满足 $c,f,s = pdefun(x,t,u,dudx)$

- pdefun 的输入参数是标量 x 和 t 以及向量 u 和 dudx。
向量 u 逼近解 u, dudx 逼近它关于 x 的偏导数。
- 输出参数 c、f 和 s 是列向量, 其元素数等于方程数。
c 存储矩阵 c 的对角线元素。

重写方程

$$1 \cdot \frac{\partial u}{\partial t} = x^0 \frac{\partial}{\partial x} (x^0 \frac{\partial u}{\partial x})$$

可知系数值 $m = 0$ 、 $c = 1$ 、 $f = \partial u / \partial x$ 和 $s = 0$ 。此方程的函数是：

```
function heatpde(x, t, u, dudx)
    return 1, dudx, 0
end
```



4.2.7 一维偏微分方程PDE

示例: $\frac{\partial u}{\partial t} = \frac{\partial^2 u}{\partial x^2}$

初始条件: $u(x,0)=1/2$

边界条件: $u(0,t)=0, u(1,t)=1$

初始条件-icfun

重写初始方程

对于 $t = t_0$ 和所有 x , 解分量均满足以下形式的初始条件:

$$u(x, t_0) = u_0(x).$$

当与参数 x 一起调用时, icfun 函数会计算并返回列向量 u_0 中 x 处的解分量的初始值。 u_0 中元素的数量等于方程的数量。

恒定初始条件 $u(x,0)=1/2$ 对应于以下函数:

```
function heatic(x)
    return 0.5
end
```

$$u_0 = \text{icfun}(x)$$



4.2.7 一维偏微分方程PDE

示例: $\frac{\partial u}{\partial t} = \frac{\partial^2 u}{\partial x^2}$

初始条件: $u(x,0)=1/2$

边界条件: $u(0,t)=0, u(1,t)=1$

边界条件-bcfun

编写函数 bcfun, 它定义边界条件的项 p 和 q。使用函数满足
 $pl,ql,pr,qr = bcfun(xl,ul,xr,ur,t)$

- ul 是左边界 $xl = a$ 处的近似解, ur 是右边界 $xr = b$ 处的近似解。
- pl 和 ql 是列向量, 对应于在 xl 处计算的 p 和 q 。同样, pr 和 qr 则与 xr 相对应。
- 输出 pl 、 ql 、 pr 和 qr 中的元素数量等于方程的数量。

当 $m > 0$ 且 $a = 0$ 时, 由于解在 $x = 0$ 附近具备有界性, 通量 f 需在 $a = 0$ 处消失。pdepe 会自动应用此边界条件, 并且忽略 pl 和 ql 中返回的值。

重写边界方程

以区间 $0 \leq x \leq 1$ 上的边界条件 $u(0,t)=0,u(1,t)=1$ 为例。
边界条件以求解器所需形式表示为

$$u(0,t) + 0 \cdot f(0,t,u,\frac{\partial u}{\partial x}) = 0,$$
$$u(1,t) - 1 + 0 \cdot f(1,t,u,\frac{\partial u}{\partial x}) = 0.$$

在这种形式下, 很明显, 两个 q 项都为零。 p 项非零, 以反映 u 的值。对应的函数是

```
function heatbc(xl, ul, xr, ur, t)
    pl = ul
    ql = 0
    pr = ur - 1
    qr = 0
    return pl, ql, pr, qr
end
```

4.2.7 一维偏微分方程PDE

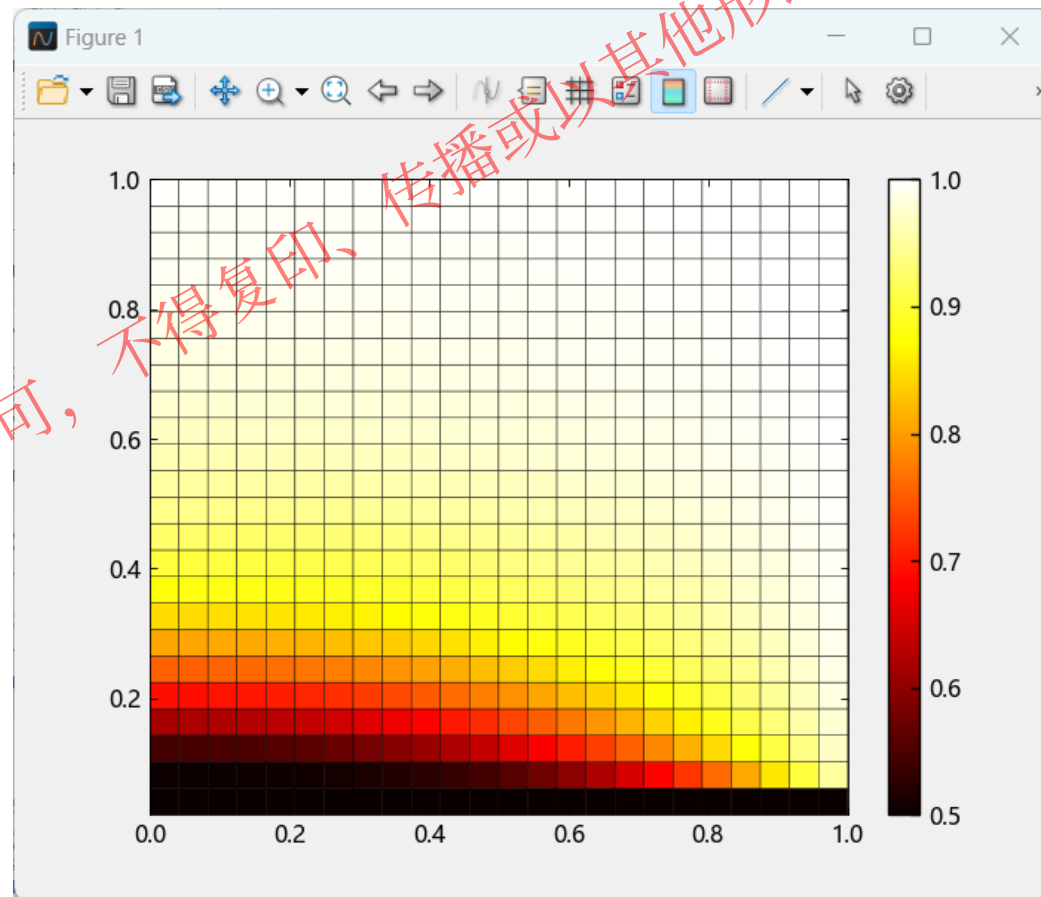
```
x = LinRange(0, 1, 25)
t = LinRange(0.02, 1, 25)
```

#求解方程

```
m=1
sol = pdepe(m, heatpde, heatic, heatbc, x, t);
u = sol[:, :, 1];
```

#可视化矩阵

```
a = pcolor(x, t, u)
colormap(a, "hot")
colorbar(gca(), a)
```



目录

1. 基础数学工具箱功能概况

2. 基础数学与线性代数

3. 随机数生成与插值

4. 数值积分与微分方程

5. 傅里叶变换与数字滤波

5.1 傅里叶变换与数字滤波

- ❑ 数字滤波板块内函数提供使用卷积对含噪二维数据进行平滑处理、用于使数据平滑或修改特定数据特性（例如信号振幅）的数据处理技术。
- ❑ 傅里叶变换类板块函数提供揭示信号的重要特征（即其频率分量）、按时间或空间采样的信号与按频率采样的相同信号进行关联的傅里叶分析函数

一. 数字滤波

数字滤波器、卷积

函数名称	功能
conv	卷积和多项式乘法
conv2	二维卷积
convn	N维卷积
deconv	去卷积和多项式除法
filter1	一维数字滤波
filter2	二维数字滤波器
padecoeff	时滞的 Padé 逼近

卷积:

两个向量 u 和 v 的卷积，表示 v 滑过 u 时依据这些点确定的重叠部分的面积。从代数方法上讲，卷积是与将其系数为 u 和 v 元素的多项式相乘相同的运算。

$m = \text{length}(u)$ 和 $n = \text{length}(v)$ 。则 w 是长度为 $m+n-1$ 且第 k 个元素为

$$w(k) = \sum_j u(j)v(k-j+1)$$

的向量。

数字滤波器:

Z 变换域中这种 filter 运算的输入-输出说明是一种有理传递函数。有理传递函数采用如下形式:

$$Y(z) = \frac{b(1)+b(2)z^{-1}+\cdots+b(n_b+1)z^{-n_b}}{1+a(2)z^{-1}+\cdots+a(n_a+1)z^{-n_a}}X(z)$$

, 可同时处理 FIR 和 IIR 滤波器。 n_a 是反馈滤波器阶数, n_b 是前馈滤波器阶数。由于归一化, 假定 $a(1) = 1$ 。

还可以将有理传递函数表示为以下差分方程:

$$a_{(1)}y_{(n)} = b_{(1)}x_{(n)} + b_{(2)}x_{(n-1)} + \cdots + b_{(n_b+1)}x_{(n-n_b)} - a_{(2)}y_{(n-1)} \cdots$$

5.1 傅里叶变换与数字滤波

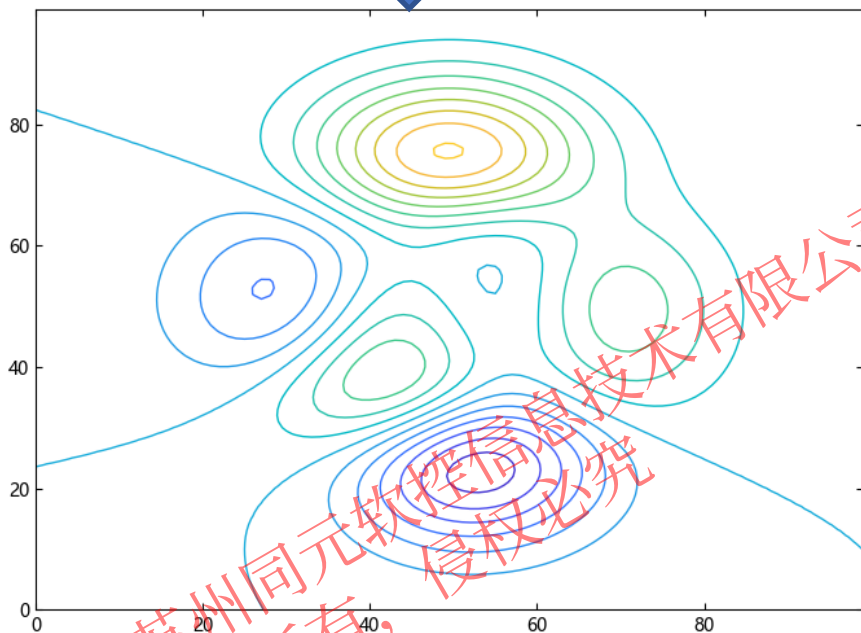
一. 数字滤波

示例1: 使用卷积对数据进行平滑处理

可以使用卷积对包含高频分量的二维数据进行平滑处理。

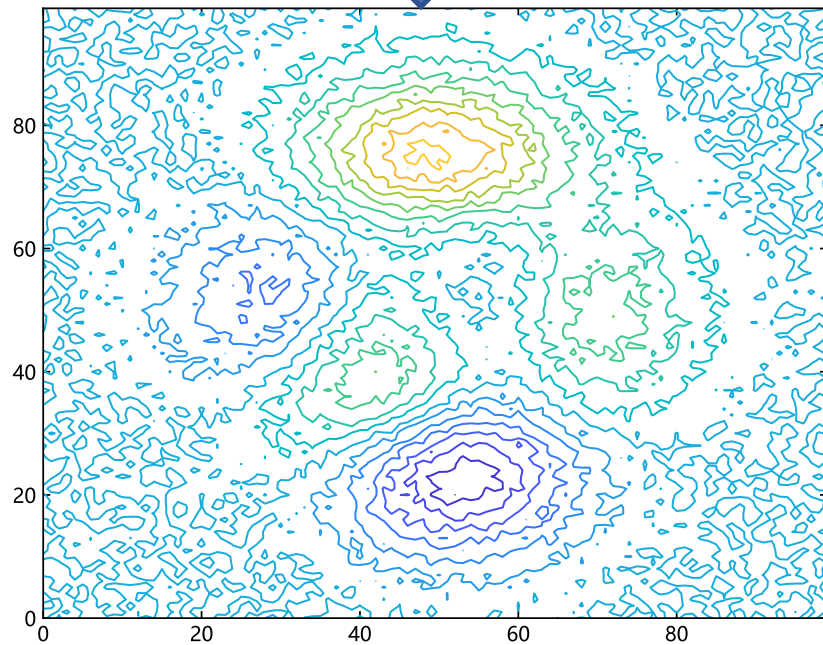
使用 peaks 函数创建二维数据，并在各个等高线层级对数据绘图

```
_,_,Z = peaks(100);  
levels = -7:1:10;  
contour(Z,levels)
```



向数据中插入随机噪声并绘制含噪等高线。

```
rng = MT19937ar(5489)  
Znoise = Z + rand(rng,100,100) .* 0.5;  
contour(Znoise, levels)
```



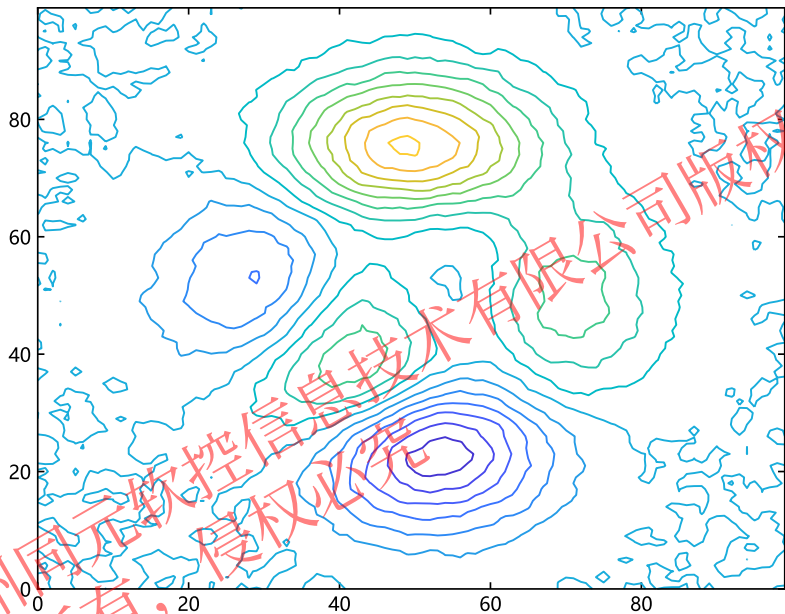
5.1 傅里叶变换与数字滤波

一. 数字滤波

示例1: 使用卷积对数据进行平滑处理

```
K = (1/9)*ones(3,3);  
Zsmooth1 = conv2(Znoise,K,"same");  
contour(Zsmooth1, levels)
```

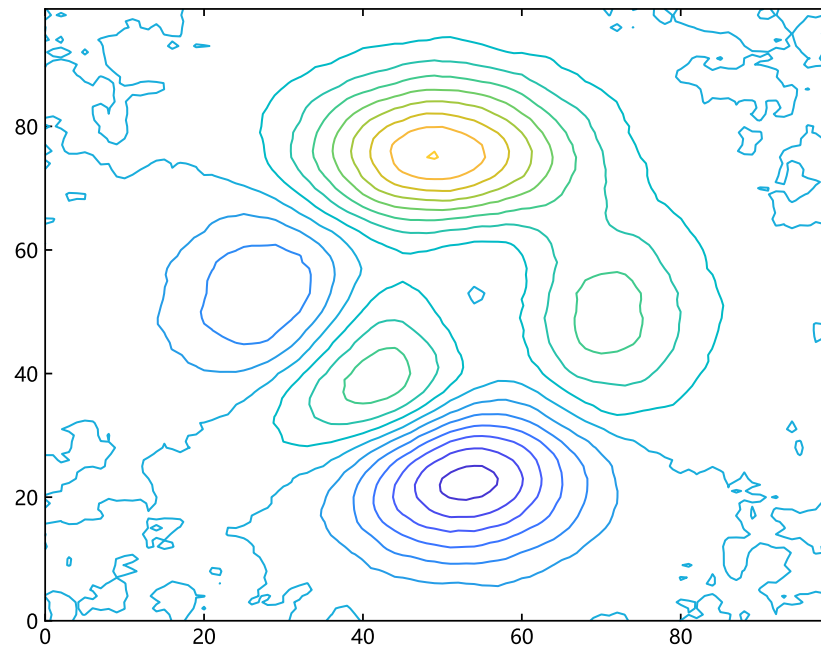
定义一个 3×3 核 K 并使用 conv2 对 Znoise 中的含噪数据进行平滑处理。绘制经过平滑处理的等高线。conv2 中的 'same' 选项使输出的大小与输入相同。



conv2 函数使用指定的核求二维数据的卷积，该核的元素定义如何去除或增强原始数据的特征。核的大小不必与输入数据相同。小核足以对仅包含少数频率分量的数据进行平滑处理。较大的核可以更精确地对频率响应进行调整，从而得到更平滑的输出。

```
K = (1/25)*ones(5,5);  
Zsmooth2 = conv2(Znoise,K,"same");  
contour(Zsmooth2, levels)
```

用 5×5 核对含噪数据进行平滑处理，并绘制新等高线。



5.2 傅里叶变换与数字滤波

- 数字滤波板块内函数提供使用卷积对含噪二维数据进行平滑处理、用于使数据平滑或修改特定数据特性（例如信号振幅）的数据处理技术。
- **傅里叶变换**类板块函数提供揭示信号的重要特征（即其频率分量）、按时间或空间采样的信号与按频率采样的相同信号进行关联的傅里叶分析函数

二. 傅里叶变换

傅里叶变换

函数名称	功能
fft	快速傅里叶变换
Nufft	非均匀快速傅里叶变换
fftshift	将零频分量移到频谱中心
ifft	快速傅里叶逆变换
Ifftshift	逆零频平移
nextpow2	2 的更高次幂的指数
interpft	一维插值 (FFT 方法)

傅里叶变换:

傅里叶变换是将按时间或空间采样的信号与按频率采样的相同信号进行关联的数学公式。在信号处理中，傅里叶变换可以揭示信号的重要特征（即其频率分量）。

对于包含 n 个均匀采样点的向量 x ，其傅里叶变换定义为

$$y_{k+1} = \sum_j \omega^{jk} x_j + 1$$

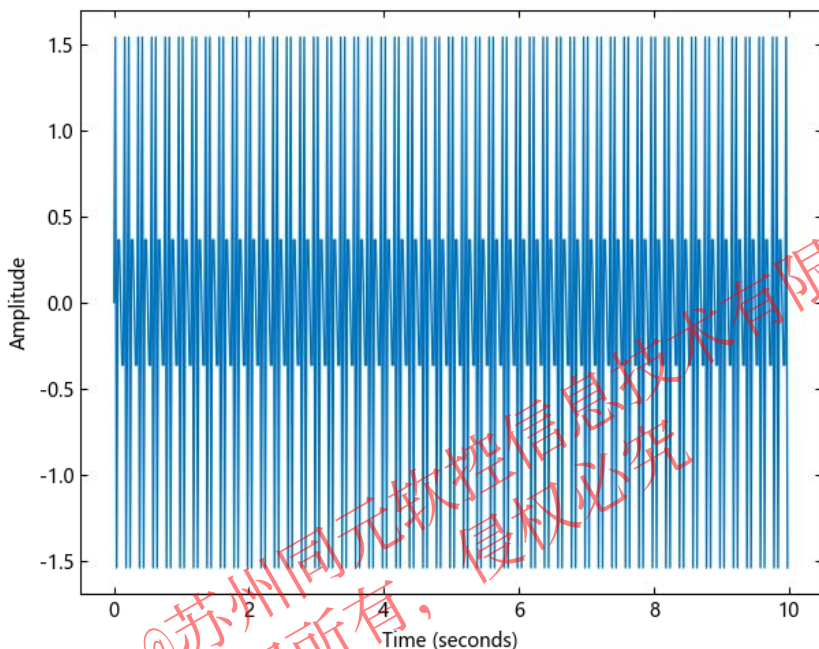
$\omega = e^{-2\pi i/n}$ 是 n 个复单位根之一，其中 i 是虚数单位。对于 x 和 y ，索引 j 和 k 的范围为 0 到 $n-1$ 。

5.2 傅里叶变换与数字滤波

二. 傅里叶变换

fft 函数使用快速傅里叶变换算法来计算数据的傅里叶变换。以正弦信号 x 为例，该信号是时间 t 的函数，频率分量为 15 Hz 和 20 Hz。使用在 10 秒周期内以 1/50 秒为增量进行采样的时间向量。

```
Ts = 1/50;  
t = 0:Ts:10-Ts;  
x = @. sin(2*pi*15*t) + sin(2*pi*20*t);  
plot(t,x)  
xlabel("Time (seconds)")  
ylabel("Amplitude")
```

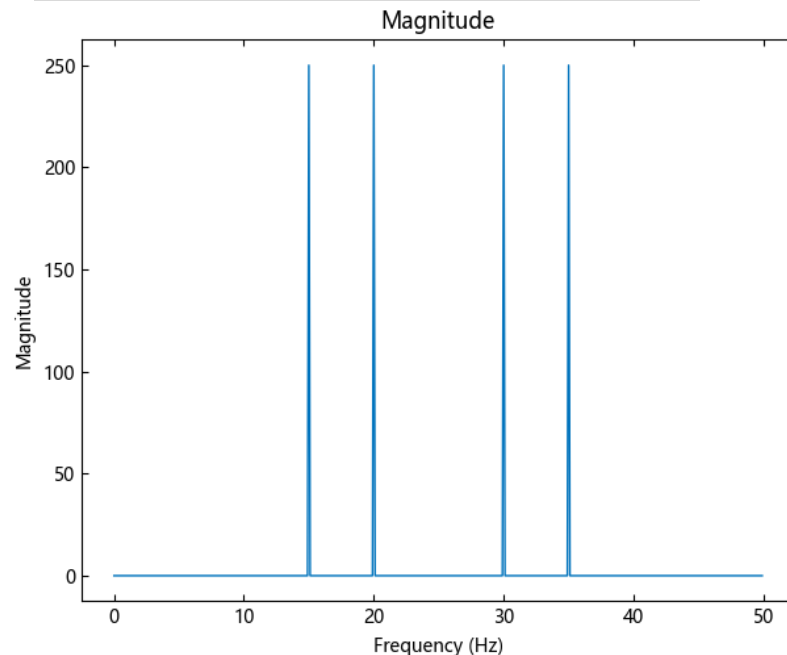


计算信号的傅里叶变换，并在频率空间创建对应于信号采样的向量 f 。

```
y = fft(x);  
fs = 1 / Ts;  
ffs = (0:length(y)-1) * fs / length(y);
```

以频率函数形式绘制信号幅值时，幅值尖峰对应于信号的 15 Hz 和 20 Hz 频率分量。

```
plot(ffs,abs.(y))  
xlabel("Frequency (Hz)")  
ylabel("Magnitude")  
title("Magnitude")
```

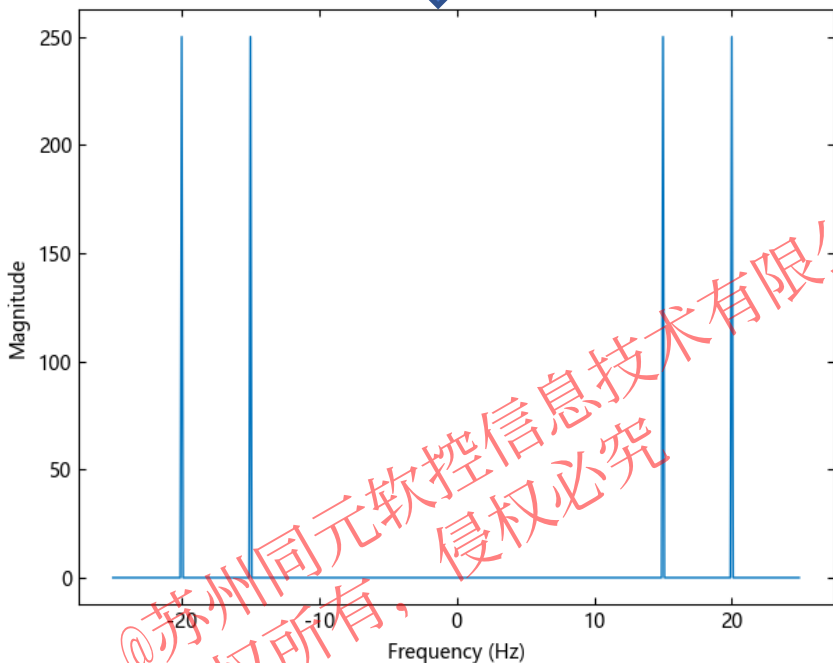


5.2 傅里叶变换与数字滤波

二. 傅里叶变换

该变换还会生成尖峰的镜像副本，该副本对应于信号的负频率。为了更好地以可视化方式呈现周期性，您可以使用 `fftshift` 函数对变换执行以零为中心的循环平移。

```
n = length(x);  
fshift = (-n/2:n/2-1)*(fs/n);  
yshift = fftshift(y);  
plot(fshift,abs.(yshift))  
xlabel("Frequency (Hz)")  
ylabel("Magnitude")
```

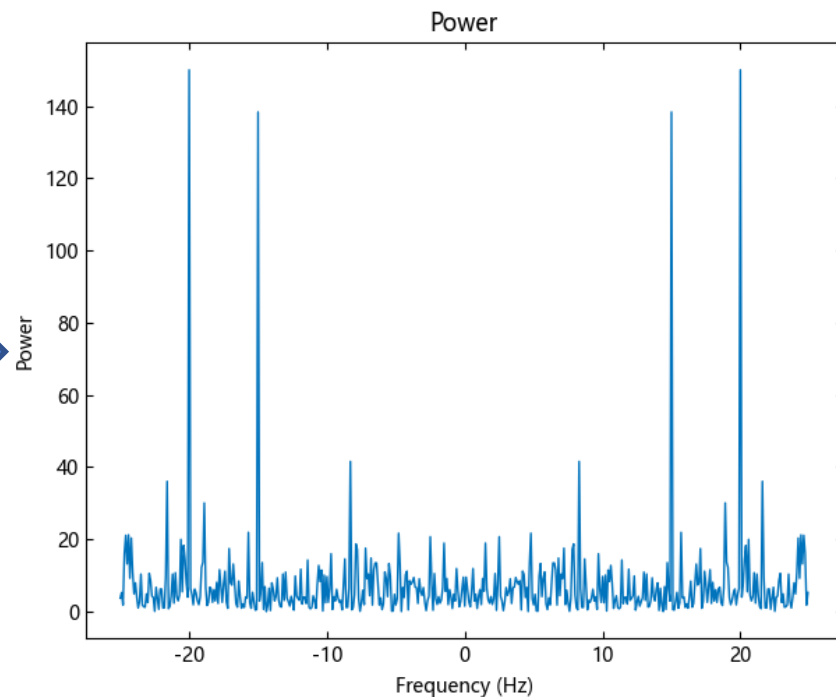


在科学应用中，信号经常遭到随机噪声破坏，掩盖其频率分量。傅里叶变换可以清除随机噪声并显现频率。例如，通过在原始信号 `x` 中注入高斯噪声，创建一个新信号 `xnoise`。

```
rng = MT19937ar(5489)  
xnoise = x + 2.5*randn(rng,size(t));
```

频率函数形式的信号功率是信号处理中的一种常用度量。功率是信号的傅里叶变换按频率样本数进行归一化后的平方幅值。计算并绘制以零频率为中心的含噪信号的功率谱。尽管存在噪声，您仍可以根据功率中的尖峰辨识出信号的频率。

```
ynoise = fft(xnoise);  
ynoiseshift =  
fftshift(ynoise);  
power = abs.(ynoiseshift).^2/n;  
plot(fshift,power)  
title("Power")  
xlabel("Frequency (Hz)")  
ylabel("Power")
```



建立知识规范，营造协同生态
积累工业模型，发展可控平台
融入中国创新，打造先进软件

感谢聆听